POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Numerical Optimization for Map-Free Navigation: Rolling-Horizon $A^\star$ and Nonlinear MPC on a Bipedal Humanoid

Project Report
062047 – Numerical Optimization for Control

Professor:
Prof. Lorenzo Mario Fagiano

Group Members:
Mateo Gomez
Lorenzo Ortolani
Francesco Pedrini

Academic year:
2025-2026

Abstract: Navigating an unknown environment without a prior map can be posed as two optimization programs solved at every control cycle: a discrete shortest-path problem on an occupancy grid rebuilt from the current LiDAR scan, and a continuous nonlinear program that turns the resulting geometric reference into a body-frame velocity command subject to the actual kinematics, the actual input limits and a clearance requirement. This report analyses that pair as optimization problems rather than as a robotics pipeline. The program deployed on a Unitree G1 humanoid is stated in full, its structure measured (141 decision variables, a 1.73% dense constraint Jacobian), and every design decision it embodies tested against the alternative the theory offers: mid-point against Euler integration, algorithmic against numerical differentiation, exact against quasi-Newton Hessians, interior-point against active-set solvers, sparse against condensed parametrisations, a smooth penalty against an exact ℓ^1 relaxation of the clearance constraint, a time-parametrised against a path-parametrised reference, and a terminal equilibrium constraint against none. The first- and second-order optimality conditions are verified on recorded missions rather than assumed. Several of the measurements contradict the design: the deployed horizon is dominated on every objective, the integrator that is 200× more accurate buys nothing in closed loop, and the constraint that decides the outcome is not the one the cost was tuned around. Every number in this document is generated from the deployed code by a single command and enters the text through macros, so the report cannot silently diverge from the software it describes.

Key-words: Nonlinear Model Predictive Control, Interior-Point Optimization, Exact Penalty Methods, Algorithmic Differentiation, Rolling-Horizon Planning, Legged Robot Navigation

Contents

Acronyms

AD	algorithmic differentiation
AMO	Adaptive Motion Optimization
DARE	discrete algebraic Riccati equation
FHOCP	finite-horizon optimal control problem
IK	inverse kinematics
IMU	inertial measurement unit
KKT	Karush–Kuhn–Tucker
LICQ	linear independence constraint qualification
LiDAR	light detection and ranging
LTI	linear time-invariant
MPC	model predictive control
NLP	nonlinear program
PD	proportional–derivative
PPO	proximal policy optimization
QP	quadratic program
RL	reinforcement learning
ROS	Robot Operating System
SQP	sequential quadratic programming

Notation

All physical quantities carry SI units. Vectors are columns; $\|\cdot\|$ without subscript is the Euclidean norm. Subscript k indexes a node inside the prediction horizon, subscript j an obstacle. The symbol $\lambda_{\min}(\cdot)$ denotes the smallest eigenvalue of its argument, and is not related to the multipliers λ.

State, input and model

x	state, Eq. (9)	τ_v, τ_w	actuator time constants [s]
u	commanded body twist	ν_v, ν_w	discrete lag coefficients, Eq. (14)
p_x, p_y, ψ	world-frame pose [m], [rad]	Δt	sampling time [s]
v_x, v_y, ω	body twist [m/s], [rad/s]	N	prediction horizon [steps]
t	continuous time [s]	N_c	control horizon [steps]

Environment and reference

$\mathcal{P}_k$	LiDAR point cloud at cycle k	δ	occupancy grid resolution [m]
$P(c)$	occupancy of cell c	σ	Gaussian inflation width [m]
x_g	goal position [m]	θ	path abscissa, Sec. 3.3.6
v_{ref}	cruise speed [m/s]	π_k	$A^\star$ path at cycle k

Optimization program

$z = (X, U)$	decision variables	λ, μ	KKT multipliers (eq., ineq.)
J	objective, Eq. (18)	μ_b	interior-point barrier parameter
Q, R, R_{jerk}	tracking, effort, rate weights	ρ_T	terminal weight multiplier
$W_{\text{obs}}, \alpha_{\text{obs}}, r_{\text{obs}}$	obstacle penalty height, steepness, radius	ρ_s	slack penalty weight
d_{safe}	imposed clearance [m]	s_{jk}	constraint slack [m]
$\gamma(k)$	robust back-off [m], Sec. 3.3.7	κ_j	scalarisation weights, Eq. (28)
ε_d	distance regulariser	ε_M	machine epsilon

Locomotion layer (Sec. 2.3)

$q, \dot{q}$	joint positions, velocities	β	gait duty factor
$\mathcal{T}$	joint torques	ζ_i	stride phase of leg i
K_P, K_D	joint PD gains	$T_{\text{str}}, T_{\text{st}}, T_{\text{sw}}$	stride, stance, swing durations [s]
p_i^{f}	foot position of leg i [m]	L_i	step length of leg i [m]

1. Introduction

A mobile robot that must reach a goal in an environment it has never seen has no global map to plan on: at every instant it knows only what its sensors reported in the last few metres. The classical answer is to decouple the problem into a geometric planner and a low-level tracker, but if the two layers ignore each other the planner produces paths the actuators cannot execute, and the tracker produces commands the geometry does not allow. Model Predictive Control (MPC) resolves the conflict by construction: it optimizes a finite sequence of future inputs subject to the actual dynamics and the actual constraints, and re-optimizes at every cycle [6, 8]. The price is computational, from the motion-planning point of view: a nonlinear program must be solved to sufficient accuracy inside the sampling period, every period, on the robot.

The compensation is architectural. Because the horizon is re-planned continuously from the current scan, the stack never has to simultaneously localize and map the environment: a localisation module — an extended Kalman filter fusing odometry and inertial measurements — is enough, and no simultaneous localisation and mapping (SLAM) back-end is required. The result is a navigation stack that is markedly less expensive to run than a map-building one, and that is deployable on a computationally modest onboard machine. Establishing what that architecture costs in optimization terms, and what it buys, is one of the contributions of this report.

The platform. The stack analysed here is deployed on a Unitree G1, a bipedal humanoid with twenty-nine actuated joints, of which the locomotion layer drives fifteen — the twelve leg joints and the three waist joints — while the arms are held at a fixed posture. None of those joints is ever addressed by the controller of this report. The robot is commanded through a single channel, a body-frame twist $u = (v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega^{\text{cmd}})$, and its only exteroceptive input is a Livox MID-360 3-D LiDAR mounted on the torso. Everything else about the machine is, by construction, invisible to the optimization program.

What produces that twist interface on a humanoid is not a gait generator but a neural network. The deployed locomotion layer is AMO (Adaptive Motion Optimization) [5], a reinforcement-learning policy trained

in simulation for the G1 and obtained by proximal policy optimization [10], with masses, friction, gains and latencies randomised during training so that the fixed policy transfers to hardware [9]. AMO is deliberately not a walking controller: it is a loco-manipulation policy, trained to track torso height, torso orientation and upper-body targets while walking. The navigation stack exercises only its locomotion subset, which is what leaves the same platform usable for manipulation later without changing the layer the MPC talks to. At each tick of its 50 Hz loop the network maps an observation — body angular velocity, gravity direction in the body frame, joint positions and velocities, previous action and the commanded twist — to an offset on the default posture, which a joint-level proportional–derivative law converts into torque. Section 2.3.2 develops this layer; Section 2.3.3 isolates the seam at which the MPC stops seeing it, because that abstraction is the hypothesis on which the entire prediction model rests: to the optimization program, twenty-nine degrees of freedom and a learned policy reduce to a first-order velocity lag.

Why a bipedal humanoid sharpens the problem. The choice of platform is not incidental to the optimization problem: it changes what the admissible input set contains, and that turns out to matter more than any weight in the cost. A wheeled base is holonomic or nearly so; a statically stable legged machine can be commanded to translate sideways and to reverse almost as freely as it advances [7]. A biped can do neither cheaply. Its policy accepts a body-frame velocity command, but lateral velocity is small and backward walking is excluded outright, both because the policy is trained forward-facing and because the perception payload — a forward-looking LiDAR — makes reversing blind. The result, formalised in Section 2.5, is an input set that is *asymmetric*: the robot may rotate and advance, essentially nothing else. Section 4.5 shows that this asymmetry, and not the obstacle term the cost was tuned around, is what binds in the situations where the mission is at risk.
A second consequence is that the prediction model and the plant are further apart than they would be on a wheeled robot. The MPC predicts with a planar model of a body that in reality balances dynamically on two feet, shifting its centre of mass at every step. Section 5.3 measures that gap on recorded missions instead of assuming it, and the measurement reorders the priorities of the whole design.

The two programs solved at every cycle. This report analyses the navigation stack as a set of optimization problems. Two are solved at every planning cycle and are the object of the analysis:

- a *discrete* shortest-path program — an $A^\star$ search [2] over a Gaussian occupancy grid rebuilt from the current LiDAR scan, whose solution is a geometric reference and whose rolling-horizon target is the intersection of the ray to the global goal with the grid boundary;
- a *continuous* nonlinear program — a six-state, three-input MPC with actuator-lag dynamics, a smooth obstacle term and box-constrained inputs, formulated once symbolically in CasADi [1] and solved by the interior-point method IPOPT [11].

The division of labour between them is deliberate and is defended in Section 3.1: the discrete program handles the combinatorial decision — which side of an obstacle to pass — and the continuous one handles feasibility and smoothness, so that the nonlinear program never has to represent a choice that would make it mixed-integer. Section 5.4 shows what happens when that separation is pushed too far, and where the boundary actually lies.

What this report does, and what distinguishes it from a system description. The stack is deployed and works; describing it would be an engineering report. What is done here instead is to take every design decision the program embodies, state the alternative that the theory of numerical optimization offers, and *measure* which one the problem actually rewards. Nine such decisions are examined: the integration scheme (Section 3.2), the parametrisation of the program (Section 3.3.1), the relaxation of the clearance constraint (Sections 3.3.2 and 3.3.3), the presence of terminal ingredients (Section 3.3.5), the parametrisation of the reference (Section 3.3.6), the tightening of the constraints against prediction error (Section 3.3.7), the computation of derivatives and of the Hessian (Section 4.2), the class of solver (Section 4.4), and the two horizons (Sections 5.5 and 5.6). The first- and second-order optimality conditions of the deployed program are verified on recorded missions rather than assumed (Section 4.5).
Several of the answers contradict the design, and those are the ones worth the space. The mid-point rule is 200× more accurate than the Euler step it replaces and buys nothing measurable in closed loop, because the prediction error is dominated by a term the integrator cannot touch (Section 5.3). The deployed horizon is dominated on every objective by a configuration a quarter of its size (Section 5.5). And the constraint that decides whether the mission succeeds is not the one the cost was tuned around: the admissible input set removes a degree of freedom that no weight can give back (Sections 4.5 and 3.3.7).

Structure. Section 2 describes the system, the platform and the environment representation, and states the navigation task as a finite-horizon optimal control problem. Section 3 derives the two programs, from the discretisation of the model to the constraints and the cost actually assembled. Section 4 concerns the numerical

machinery: dimensions and sparsity, derivatives, choice of solver, and the optimality conditions on the deployed instance. Section 5 reports the measurements, one per design decision. Sections 6 and 7 discuss the limitations and conclude.

Reproducibility as a property of the document. Every number in this report — in the text, in the tables and in the figures — is produced by a single command run against the deployed code, and enters the document through generated macros rather than by being typed. The measurements in this version were taken on commit `a54dfe30ec` of branch `G1_optimal_trajectory` (2026-08-24), with the profile `planner_params_g1.yaml` and the recorded run `industrial_plant_v3`, using CasADi 3.7.2 and NumPy 1.26.4. Re-running the campaign and recompiling updates the report; a parameter changed in the code and not in the text cannot go unnoticed, because there is no text to change.

2. Problem analysis

2.1. System overview

The stack is a chain of Robot Operating System (ROS) 2 nodes closing the loop between a 3-D LiDAR and the body-frame velocity interface of the locomotion controller:

1. a **pose source**, which converts the state estimate into a single pose topic and differentiates it, with an exponential filter, into a body-frame velocity estimate — the platform does not publish a usable body-frame twist;
2. a **mapping module**, which rebuilds a local Gaussian occupancy grid from the newest scan at 1 Hz and maintains an obstacle memory in the world frame, whose purpose is precisely to repair the failure mode analysed in Section 5.8;
3. the $A^\star$ **local planner**, which solves the discrete program of Section 3.1 on that grid;
4. the **nonlinear MPC tracker**, which solves the nonlinear program (NLP) of Section 3.3 and publishes a setpoint;
5. a **setpoint-to-twist stage**, a proportional law on a look-ahead point of the predicted trajectory, which produces the `/cmd_vel` actually sent to the locomotion layer.

Two design decisions shape everything that follows. First, the locomotion layer is treated as an ideal velocity-tracking device with a time constant: the optimization never models legs, contacts or gait, and the robot is abstracted as a planar holonomic agent. Sections 2.2 and 2.3 describe what that layer actually does on each of the two platforms and why the same abstraction survives the difference between them. Second, the map is *local and volatile*: the occupancy grid covers a fixed window centred on the robot and is rebuilt from scratch at every planning cycle, so the planning problem is genuinely map-free and the geometric reference the MPC tracks is only valid for one grid width ahead.

Point 5 deserves emphasis, because it is a deviation from the textbook receding-horizon implementation and it is not innocuous. The optimizer is used as a *reference generator* rather than as a direct controller: $u_0^\star$ is not applied to the plant. The consequence is measured in Section 3.3.7, where an improvement obtained in the plan turns out not to be observable in the executed trajectory, and is the strongest argument for closing the loop as the theory prescribes.

2.2. The controlled platforms

The same stack drives two machines whose mechanics could hardly be more different: a Unitree G1 humanoid, which is the platform of this report, and a Unitree Go2 quadruped, on which the stack was originally developed. The Go2 has twelve actuated joints, three per leg; the G1 has twenty-nine, of which the locomotion layer of Section 2.3.2 drives fifteen — the twelve leg joints and the three waist joints — while the arms are held at a fixed posture. Neither joint set is ever addressed by the controller of this report. Both platforms are commanded through the same channel, a body-frame twist $u = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega^{\mathrm{cmd}})$, and both carry a 3-D LiDAR whose returns are the only exteroceptive input of the stack.

Table 1 collects what distinguishes them from the controller's point of view. The last two rows are the only ones that enter the optimization program, and they are where the two profiles genuinely differ: the admissible input set and the actuator time constant.

The lateral bound is the entry to keep in mind for the rest of the report. A quadruped strafes; a biped does not, and the G1 profile sets $v_{y,\max} = 0.02$ m/s — not exactly zero, only for the conditioning of the solver. Section 4.5 shows that this does not *limit* a degree of freedom so much as *remove* it, and Section 3.3.7 shows the consequence: a state constraint that asks the robot to move away from a wall it is already close to cannot

Table 1: The two platforms as the controller sees them. The rows above the rule are properties of the machine, those below are properties of the profile that the optimization program actually reads. The deployed G1 values are quoted from the generated macros, so they cannot drift from the parameter file.

	Unitree Go2	Unitree G1
Morphology	quadruped	humanoid
Actuated joints	12 (3 per leg)	29 (15 driven by the policy)
Exteroception	Unitree L1 LiDAR	Livox MID-360 LiDAR
Locomotion layer	CHAMP gait controller	AMO policy, Sec. 2.3.2
Layer update rate	200 Hz	50 Hz
Simulator	Gazebo, dynamic	MuJoCo, kinematic
Input envelope (v_x, v_y, ω)	1.0, 0.5, 1.5	0.30, 0.02, 0.30
Actuator lag τ	0.12 s / 0.10 s	0.00100 s (degenerate, see Section 2.5)
Horizon N, Δt	50, 0.1 s	15, 0.20 s

be satisfied, because there is no admissible input that translates the body sideways.

2.3. Low-level locomotion layers

The controller analysed in this report never commands a joint. Its decision variable is a body-frame twist, and between that twist and the ground sits a platform-specific layer that turns it into joint commands at 200 Hz on the Go2 and 50 Hz on the G1. Only two properties of that layer reach the optimization problem: the twist is tracked with a first-order lag rather than instantaneously, and the tracking is fast enough, relative to the MPC cycle, for the two layers to be designed separately. This section describes the two locomotion controllers to the depth needed to justify those two properties, and no further. Throughout, $q \in \mathbb{R}^{n_j}$ denotes the vector of joint positions ($n_j = 12$ on the Go2, $n_j = 29$ on the G1), $\mathcal{T} \in \mathbb{R}^{n_j}$ the vector of joint torques, and $K_P, K_D \succ 0$ the diagonal gain matrices of the joint-level loop; the symbol τ is reserved for the actuator time constants of Section 2.5.

2.3.1 Model-based gait generation: CHAMP on the Go2

CHAMP [3] is an open-source implementation of the hierarchical trot controller developed for the MIT Cheetah [4]. It is a purely feedforward pipeline: a pattern modulator imposes the footfall sequence, a trajectory planner turns each leg's phase into a foot position, inverse kinematics turns that position into joint angles, and a joint-level impedance loop tracks them.

Pattern modulation. Each leg $i \in \{\mathrm{LF}, \mathrm{RF}, \mathrm{LH}, \mathrm{RH}\}$ carries a normalised stride clock

$$\zeta_i(t) = \left(\frac{t}{T_{\mathrm{str}}} - \zeta_i^0 \right) \bmod 1 \in [0, 1), \qquad T_{\mathrm{str}} = T_{\mathrm{st}} + T_{\mathrm{sw}}, \tag{1}$$

where T_{st} and T_{sw} are the stance and swing durations and ζ_i^0 the phase offset of leg i. Leg i is in stance while $\zeta_i < \beta$ and in swing otherwise, with duty factor $\beta = T_{\mathrm{st}}/T_{\mathrm{str}}$. The deployed configuration uses $T_{\mathrm{st}} = T_{\mathrm{sw}} = 0.25$ s, hence $T_{\mathrm{str}} = 0.5$ s and $\beta = 0.5$, with $\zeta^0 = (0, \frac{1}{2}, \frac{1}{2}, 0)$: the diagonal pairs move together, which is the trot.

Where the command enters. The commanded twist does not act on the legs directly; it acts on the *step geometry*, through the Raibert foot-placement heuristic [7]. A foot is placed half a step ahead of its nominal position, so that sweeping backwards during the stance interval T_{st} displaces the trunk by exactly the commanded amount. With $\bar{p^{\mathrm{f}}}_i \in \mathbb{R}^3$ the nominal position of foot i in the body frame,

$$\Delta p_i^{\mathrm{f}} \;=\; R_z(\chi_g)\Big(\bar{p^{\mathrm{f}}}_i + \tfrac{1}{2} T_{\mathrm{st}} \left(v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0\right)^\top\Big) - \bar{p^{\mathrm{f}}}_i, \qquad \chi_g \approx \tfrac{1}{2} T_{\mathrm{st}}\, \omega^{\mathrm{cmd}}, \tag{2}$$

with $R_z(\cdot)$ the elementary rotation about the vertical axis and χ_g the yaw swept in half a stance interval. The step length and the heading of the foot trajectory follow as

$$L_i = 2\left\|\Delta p_i^{\mathrm{f}}\right\|_2, \qquad \chi_i = \operatorname{atan2}\left(\Delta p_{i,y}^{\mathrm{f}},\, \Delta p_{i,x}^{\mathrm{f}}\right). \tag{3}$$

Equations (2)–(3) are the whole of the command path: the three numbers the MPC publishes are compressed into one length and one angle per leg, and nothing else of the command reaches the robot.

Foot trajectories and impedance. With phase and step length fixed, each foot follows a planar curve in its own trajectory frame, rotated by χ_i and added to the nominal stance. In stance, parametrised by $\zeta_i^{\mathrm{st}} = \zeta_i/\beta$, the foot sweeps backwards along a shallow arc that presses into the ground by h_{st}; in swing it follows a Bézier curve of degree 11 whose vertical spread is the swing height h_{sw}. A closed-form inverse kinematics (IK) map takes each foot position to the three joint angles of its leg, $q_i^{\mathrm{ref}} = \mathrm{IK}_i(p_i^{\mathrm{f}})$, and a proportional–derivative law closes the loop at the joints,

$$\mathcal{T}_i = K_P\big(q_i^{\mathrm{ref}} - q_i\big) + K_D\big(\dot{q}_i^{\mathrm{ref}} - \dot{q}_i\big) \;=\; \boldsymbol{J}_i(q_i)^{\top} F_i, \tag{4}$$

where $\boldsymbol{J}_i$ is the foot Jacobian of leg i and F_i the equivalent force at the foot. Read from right to left, (4) is the point of the architecture: a joint-space proportional–derivative (PD) loop realises a virtual spring–damper at the foot, of Cartesian stiffness $\boldsymbol{J}_i^{-\top} K_P \boldsymbol{J}_i^{-1}$, so the leg absorbs terrain irregularity and impact by programmable compliance rather than by contact sensing or force control. This is what lets an open-loop gait generator survive on a real robot.

Why this layer looks like a lag. Three consequences propagate upstream, and they are the reason the model of Section 2.5 has the form it has. First, the twist enters only through the step geometry: no measured trunk velocity is fed back, so the layer is a feedforward map and the trunk velocity approaches the commanded one over roughly one stride rather than immediately. Second, that step geometry is re-evaluated once per stride, so a command changing faster than $1/T_{\mathrm{str}} = 2$ Hz is averaged, not tracked; a first-order lag with $\tau_v = 0.12$ s reaches 98% of its final value in $4\tau_v \approx T_{\mathrm{str}}$, which is why a one-parameter fit describes the layer as well as it does. Third, the gait applies its own saturation, which is where the Go2 input envelope of Table 1 comes from.

2.3.2 Learned whole-body control: the AMO policy on the G1

On the humanoid the same interface is produced by a neural network instead of a gait generator. The reference controller is AMO (Adaptive Motion Optimization) [5], a reinforcement-learning policy trained in simulation for the G1 and its twenty-nine degrees of freedom. AMO is deliberately *not* a walking controller: it is a loco-manipulation policy, trained to track torso height, torso orientation and upper-body targets while walking, on a hybrid dataset in which an offline trajectory-optimization module supplies whole-body references kinematically consistent with the commanded end-effector motion. The navigation stack exercises only its locomotion subset — the arms are held at the default posture and only the base command is driven — but the policy is the one trained for the larger task, which is what would make the same platform usable for manipulation later without changing the layer the MPC talks to.

Structure. At each tick k of the 50 Hz policy loop, the network maps an observation to an action,

$$a_k = \pi(o_k) \in \mathbb{R}^{15}, \qquad o_k = \big(\,\omega_k^{\mathrm{b}},\; \hat{g}_k,\; \tilde{u}_k,\; q_k - q^{\mathrm{def}},\; \dot{q}_k,\; a_{k-1}\,\big), \tag{5}$$

where $\omega_k^{\mathrm{b}} \in \mathbb{R}^3$ is the body angular velocity, $\hat{g}_k \in \mathbb{R}^3$ the gravity direction in the body frame (the tilt an inertial measurement unit, IMU, provides without a global heading), $q_k, \dot{q}_k$ the measured joint positions and velocities, q^{def} the default posture, a_{k-1} the previous action, and $\tilde{u}_k$ the command. The action is not a torque: it is an offset on the default posture, which the same joint-level PD law as (4) converts into torque,

$$q_k^{\mathrm{ref}} = q^{\mathrm{def}} + s_a\, a_k, \qquad \mathcal{T}_k = K_P\big(q_k^{\mathrm{ref}} - q_k\big) + K_D\big(0 - \dot{q}_k\big), \tag{6}$$

with s_a a fixed action scale. The learned part of the controller is therefore exactly the map from observation to PD setpoint; the stabilising loop underneath it is the same fixed impedance as on the quadruped. The policy is obtained by proximal policy optimization (PPO) [10], maximising a discounted return whose stage reward pays for tracking the commanded twist and penalises posture deviation, joint effort and action rate, with masses, friction, gains and latencies randomised during training so that the fixed policy transfers to hardware.

The command mapping, and one unmodelled nonlinearity. The command handed to the network is not the MPC twist but its image under a calibrated map, $\tilde{u}_k = \mathcal{M}(u_k)$, and that map is not a change of units: the policy's yaw channel is a heading *offset* rather than a yaw rate, its lateral channel uses the opposite sign convention, and the policy plants its feet whenever all three channels fall below a stand gate. That gate is the one visible unmodelled nonlinearity of the cascade — a near-pure rotation in place, which the MPC produces when the reference turns sharply at low speed, falls below it and returns no motion at all. It is a discontinuity in the map from the optimizer's output to the robot's motion, and it is not represented anywhere in the program.

2.3.3 The cascade, its abstraction seam, and what it costs

The stack is a cascade of four loops whose rates are separated by roughly a decade each: the $A^\star$ reference is recomputed at 1 Hz, the MPC at 1/0.20 Hz, the locomotion layer at 50–200 Hz, and the joint PD loop at the motor-driver rate. That separation is the standard justification for designing each layer against a static abstraction of the one below it, and the abstraction used here is deliberately minimal: three decoupled first-order lags, Eq. (10). Both locomotion layers, the hand-designed one and the learned one, are approximated by the same two numbers.

It is worth stating what that abstraction discards, because the report returns to it twice. It discards leg dynamics, contact, footstep timing, the gait saturation of Section 2.3.1 and the stand gate of Section 2.3.2. What survives is a question of degree, and Section 5.3 answers it quantitatively: measured against a recorded mission, the prediction diverges from the executed trajectory by 0.305 m over one horizon — 18 times the truncation error of the integration scheme it replaced, and 3505 times that of the scheme now deployed. The abstraction, not the arithmetic, is the dominant error term of the whole prediction.

2.4. Environment representation

Let $\mathcal{P}_k = \{p_i \in \mathbb{R}^3\}_{i=1}^{N_k}$ be the LiDAR point cloud available at cycle k, projected on the plane. The grid is a square of side $2h_w$ centred on the robot position and discretised with resolution Δ. The occupancy probability of cell c is a Gaussian tail of the distance to the nearest return,

$$P(c) \;=\; 1 - \Phi\!\left(\frac{d_{\min}(c,\mathcal{P}_k)}{\sigma}\right), \qquad d_{\min}(c,\mathcal{P}_k) = \min_{p\in\mathcal{P}_k} \|c-p\|_2, \tag{7}$$

with Φ the standard normal cumulative distribution function. Equation (7) is an *inflation* model: it is not a Bayesian occupancy update — there is no recursive filtering, no ray casting and no free-space carving at this stage — but a smooth, monotone map from clearance to cost, whose one parameter σ sets the width of the obstacle halo.

Cells with $P(c) \geq P_{\text{thr}}$ are removed from the search graph. The pair (σ, P_{thr}) is not tuned by hand: inverting (7) gives the hard-blocking radius implied by a threshold,

$$d_{\text{block}} \;=\; \sigma\,\Phi^{-1}(1-P_{\text{thr}})\,, \tag{8}$$

and the G1 profile chooses σ and P_{thr} so that $d_{\text{block}} \approx 0.40$ m against a body radius of about 0.35 m. This is a small point but a representative one: the parameter that matters is a physical clearance, and the two grid parameters are derived from it rather than chosen and justified afterwards.

Because the grid is rebuilt from scratch, its construction cost is the dominant cost of the planning layer, $O(n_c^2\,|\mathcal{P}_k|)$; vectorised, it is negligible against the MPC at the 1 Hz replanning rate.

2.5. Plant model and state-space formulation

The MPC model must predict where the robot will be, not where it was told to go. Commanding a legged platform a body-frame velocity does not, in general, produce that velocity instantaneously: the locomotion controller responds with a time constant, which over a horizon accumulates into a prediction error if neglected. The model therefore augments the planar kinematic state with the three velocity states,

$$x \;=\; \begin{bmatrix} p_x & p_y & \psi & v_x & v_y & \omega \end{bmatrix}^\top \in \mathbb{R}^6, \qquad u \;=\; \begin{bmatrix} v_x^{\text{cmd}} & v_y^{\text{cmd}} & \omega^{\text{cmd}} \end{bmatrix}^\top \in \mathbb{R}^3, \tag{9}$$

where (p_x, p_y) and ψ are the world-frame pose and (v_x, v_y, ω) the body-frame twist actually realised. In continuous time the actuator behaves as three decoupled first-order lags driven by the commands,

$$\dot v_x = \frac{u_1 - v_x}{\tau_v}, \qquad \dot v_y = \frac{u_2 - v_y}{\tau_w}, \qquad \dot\omega = \frac{u_3 - \omega}{\tau_w}, \tag{10}$$

while the pose obeys the holonomic body-to-world rotation

$$\dot p_x = v_x\cos\psi - v_y\sin\psi, \qquad \dot p_y = v_x\sin\psi + v_y\cos\psi, \qquad \dot\psi = \omega. \tag{11}$$

Model (10)–(11) is holonomic: unlike a differential-drive vehicle, the platform can in principle command a lateral velocity, so no nonholonomic constraint appears and the input set is a box. This is what makes the resulting NLP comparatively benign — the nonlinearity is confined to the rotation $R(\psi)$ and to the obstacle distances.

On the deployed time constant, and why it is degenerate. The G1 profile sets $\tau_v = \tau_w = 0.00100$ s. This is not an identified value and is not meant to be one: the MuJoCo plant used throughout this report integrates the commanded twist *kinematically*, with no actuation dynamics at all, so the discrete lag coefficient of Section 3.2 evaluates to $1 - e^{-\Delta t/\tau} = 1.000000$ and the velocity update collapses to $v_{k+1} = u_k$. The choice is deliberate and its purpose is methodological: with the prediction model equal to the simulation model by construction, the experiments of Sections 4 and 5 measure the behaviour of the *optimizer* rather than the noise of a gait. The cost of the choice is that any conclusion which depends on the actuator pole — the selection of the sampling time from the plant bandwidth being the obvious one — cannot be drawn from these data, and is deferred until τ is identified under physics. Section 3.3.5 shows one place where the degeneracy visibly flatters a result, and says so.

2.6. The navigation task as a finite-horizon optimal control problem

Let $x_g \in \mathbb{R}^2$ be the goal. Written in the standard form, the task is the finite-horizon optimal control problem

$$\begin{aligned}
\min_{u_0,\dots,u_{N-1}} \quad & J(x_{0:N}, u_{0:N-1}, \mathcal{P}_k, \theta) && && \text{(12a)}\\
\text{s.t.} \quad & x_{t+1} = f(x_t, u_t), && t = 0, \dots, N-1, && \text{(12b)}\\
& x_0 = \hat{x}(t_k), && && \text{(12c)}\\
& u_t \in \mathcal{U}, && t = 0, \dots, N-1, && \text{(12d)}\\
& d(x_t, \mathcal{P}_k) \geq r_{\min}, && t = 1, \dots, N, && \text{(12e)}\\
& x_N \in \mathcal{X}_f, && && \text{(12f)}
\end{aligned}$$

solved at every sampling instant t_k, of which only u_0 is retained. Three features of (12) determine the whole design, and each is taken up in turn.

- **The goal is not reachable within the horizon.** With $N\Delta t = 3.0$ s and a cruise speed of 0.20 m/s the horizon spans well under a metre of travel, while the goal may be tens of metres away and outside the sensed region altogether. The cost (12a) therefore cannot be a distance to x_g; it must be a distance to a *reference trajectory* produced by a separate, cheaper program that looks further ahead. That program is the $A^\star$ search of Section 3.1.
- **The obstacle set is a point cloud, not a polytope.** Constraint (12e) is non-convex and its description changes at every cycle. The deployed program relaxes it into a smooth penalty (Section 3.3.2); Section 3.3.3 re-poses it as a genuine constraint with a slack and shows at what penalty weight the relaxation becomes exact.
- **The terminal constraint is optional and its cost is measurable.** Constraint (12f) is absent from the deployed configuration (`none`). Section 3.3.5 switches it on, verifies that the terminal set is reachable at every operating point tested, and prices it.

Note the index range of (12e), which starts at $t = 1$ and not at $t = 0$. At the first node the state is fixed by the equality (12c), so a constraint imposed there does not depend on any decision variable: it cannot change the solution, and it renders the entire program infeasible whenever the measured state already violates it — which is routine, since the robot is frequently closer to an obstacle than the safety radius nominally allows. This is not a detail of implementation but a property of the formulation, and getting it wrong is a documented way to make a well-posed problem unsolvable.

3. Formulation of the optimization program

3.1. Reference generation: $A^\star$ as a discrete program

At every planning tick the geometric reference is the solution of a shortest-path problem on the 8-connected grid of Section 2.4. Cells above the occupancy threshold are removed from the graph; the remaining edges carry the cost

$$g(n \to n') \;=\; c_{\text{move}} \cdot \delta \cdot \left[1 + w_{\text{obs}} \left(\frac{P(n')}{P_{\text{thr}}} \right)^2 \right], \qquad c_{\text{move}} \in \{1, \sqrt{2}\}. \tag{13}$$

The normalised quadratic in (13) is the mechanism that produces medial-axis-like paths: a cell just below the hard threshold costs $1 + w_{\text{obs}}$ times an open cell, so the search prefers the middle of a corridor over the short path that grazes a wall, yet a doorway is never closed outright. The Euclidean heuristic is admissible because the bracket in (13) is bounded below by 1, so no edge can be cheaper than its Euclidean length; the search is therefore an exact $A^\star$ and returns a cost-optimal path on the current grid.

Why this matters for the continuous program. The decision of *which side* of an obstacle to pass on is combinatorial. Posed inside the NLP it would require binary variables — a mixed-integer nonlinear program, whose solution time is not compatible with a control loop. Delegating it to a graph search on a discretised free space is what keeps the continuous program a smooth NLP with a box-shaped input set, and it is the single most consequential structural decision in the stack. The price is that the discrete layer commits to a homotopy class before the continuous layer is consulted, and the continuous layer has no way to revise that commitment: Section 5.4 measures what happens when the smooth program is left with two competing basins of its own, and Section 5.8 shows the failure that follows when the discrete layer commits on the basis of a memory it does not have.

3.2. Model discretization

With sampling time Δt, the lag subsystem (10) is linear and time-invariant, so it is discretised *exactly* under a zero-order-hold input. Defining $\nu_v = 1 - e^{-\Delta t/\tau_v}$ and $\nu_w = 1 - e^{-\Delta t/\tau_w}$, the velocity update is the exact solution of (10) over one step,

$$v_{x,k+1} = (1-\nu_v)v_{x,k} + \nu_v u_{1,k}, \qquad \omega_{k+1} = (1-\nu_w)\omega_k + \nu_w u_{3,k}, \tag{14}$$

and analogously for v_y. No approximation is involved here and none should be claimed: this is the Lagrange formula for a linear time-invariant (LTI) system under a piecewise-constant input, and it is exact at the sampling instants. On the deployed G1 profile it is also *vacuous*, since $\nu = 1.000000$ reduces (14) to $v_{k+1} = u_k$; the statement is retained because it is the part of the model that would carry the weight on hardware, where τ is real.

The pose, by contrast, is genuinely nonlinear and must be integrated numerically. Two schemes are available in the deployed code, selected by a parameter:

$$\text{explicit Euler:} \quad p_{k+1} = p_k + R(\psi_k)\, v_{k+1}\, \Delta t, \tag{15}$$

$$\text{mid-point (RK2):} \quad p_{k+1} = p_k + R\big(\psi_k + \tfrac{1}{2}\omega_{k+1}\Delta t\big)\, v_{k+1}\, \Delta t. \tag{16}$$

Both evaluate the rotation on the *post-lag* velocity v_{k+1}, a semi-implicit choice that costs nothing and removes the one-step delay an explicit scheme would introduce between a command and the predicted displacement. The mid-point rule differs from Euler by one addition inside the argument of the rotation.

The measured order. The global error of each scheme was fitted against the step size on three motion regimes, with the exact solution available in closed form (for $\omega \neq 0$ the displacement is a circular arc). The measured orders are 1.00 and 2.00, i.e. exactly the first and second order the truncation analysis predicts, and the result is identical across regimes.

Table 2: Fitted global truncation order of the two integrators, by motion regime (log–log fit of the global error against the step size).

motion regime	order, Euler	order, mid-point
nominal ($v_x = 0.2$, $\omega = 0.3$)	1.00	2.00
with lateral drift	1.00	2.00
fast rotation ($\omega = 1.0$)	1.00	2.00

Table 3: Global error against the step size, nominal ($v_x = 0.2$, $\omega = 0.3$) regime. The orders of Table 2 are the log–log slopes of these two columns.

Δt [s]	error, Euler [m]	error, mid-point [m]
0.2000	1.74×10^{-2}	8.70×10^{-5}
0.1000	8.70×10^{-3}	2.17×10^{-5}
0.0500	4.35×10^{-3}	5.44×10^{-6}
0.0250	2.17×10^{-3}	1.36×10^{-6}
0.0125	1.09×10^{-3}	3.40×10^{-7}

At the deployed $\Delta t = 0.20$ s and over a three-second window, Euler is wrong by 1.74×10^{-2} m and the mid-point rule by 8.70×10^{-5} m, a factor 200 for the price of one addition. The test also verifies that the dynamics

assembled *inside* the NLP coincides with the reference scheme to machine precision: it is not enough for the theory to check out, the tracker must integrate that way.
Section 5.3 returns to this and reaches an uncomfortable conclusion, which is why the two sections should be read together: a factor 200 in the accuracy of the integrator changes the closed loop by essentially nothing, because the integrator was never the dominant error term.

3.3. The nonlinear MPC program

3.3.1 Decision variables, cost, and the parametrisation of the program

The NLP is written in the *simultaneous* form, also called direct multiple shooting: both the state and the input sequences are decision variables and the dynamics enter as equality constraints,

$$z \;=\; \big(X, U\big), \qquad X = [x_0, \dots, x_N] \in \mathbb{R}^{6\times(N+1)}, \quad U = [u_0, \dots, u_{N-1}] \in \mathbb{R}^{3\times N}. \tag{17}$$

With $e_k = x_k - x_{\mathrm{ref},k}$ the cost is

$$J(z;\theta) \;=\; \sum_{k=0}^{N-1} \Big[e_k^\top Q\, e_k \;+\; u_k^\top R\, u_k \;+\; R_{\mathrm{jerk}} \|u_k - u_{k-1}\|_2^2 \;+\; J_{\mathrm{obs}}(x_k) \Big] \;+\; e_N^\top Q_T\, e_N + J_{\mathrm{obs}}(x_N), \tag{18}$$

with $Q = \mathrm{diag}(Q_x, Q_y, Q_\psi, 0, 0, 0)$, $Q_T = \rho_T\, Q$ and $R = \mathrm{diag}(R_{v_x}, R_{v_y}, R_\omega)$. Two properties deserve comment. The stage weight is only *semi*-definite: no velocity state is penalised, Q has rank 3 out of 6, and the velocities are shaped indirectly through the lag dynamics and the input penalty. And the rate penalty $R_{\mathrm{jerk}}\|u_k - u_{k-1}\|^2$ is the only term coupling adjacent input stages, which is what puts off-diagonal blocks in the Hessian; its purpose is physical, since a legged platform tracks a smooth velocity profile far better than a chattering one.

The alternative, and what it actually costs. The standard argument for multiple shooting is that it keeps the Jacobian sparse and banded, where a condensed single-shooting formulation — states eliminated by recursive substitution, inputs as the only variables — would produce a small but dense problem. The deployed code implements both, with the transition map written once and shared, so what is compared is the parametrisation and not two different models.

Table 4: Multiple (M) against single (S) shooting on the same instance. Timings are single cold solves; dimensions and densities are exact.

N	vars M / S	cons M / S	jac dens. [%] M / S	hess dens. [%] M / S	solve [ms] M / S
5	51 / 15	56 / 20	4.59/6.67	12.11/100.00	86 / 147
15	141 / 45	156 / 60	1.73/2.22	4.45/100.00	209 / 191

The structural half of the claim holds exactly, and the Hessian is where it shows: at $N = 15$ the program shrinks from 141 variables and 156 constraints to 45 and 60, while the Hessian density goes from 4.45% to 100.00% — a full matrix. Condensing does trade a large banded problem for a small dense one.
The performance half does not follow from that, and it would be dishonest to claim it does on these data. Each timing in Table 4 is a single cold solve, so it carries the variance of one sample while the dimensions beside it are exact; and on the horizons tested the condensed form is not uniformly slower. What can be asserted without a stopwatch is the conditioning argument: single shooting integrates the model in *open loop* over the whole horizon, so the error compounds step by step and the problem degrades with N and with any instability of the plant. On a kinematic, stable model that defect does not surface — which is precisely why this comparison cannot be used to argue the general case, and why the right claim to make is about structure rather than speed.

3.3.2 Obstacle avoidance as a smooth penalty

The clearance constraint (12e) is replaced, in the deployed configuration, by a penalty evaluated on the M nearest LiDAR returns $\{p_j\}$ inside a search radius:

$$J_{\mathrm{obs}}(x_k) \;=\; W_{\mathrm{obs}} \sum_{j=1}^{M} \Bigg[\underbrace{\tfrac{1}{2}\Big(1 - \tanh\big(\tfrac{\alpha_{\mathrm{obs}}}{2}(d_{k,j} - r_{\mathrm{obs}})\big)\Big)}_{\text{sigmoid zone}} \;+\; \underbrace{2\max\big(0,\; r_{\mathrm{obs}} - d_{k,j}\big)^2}_{\text{penetration penalty}} \Bigg], \tag{19}$$

with $d_{k,j} = \sqrt{(p_{x,k} - p_{j,x})^2 + (p_{y,k} - p_{j,y})^2 + \varepsilon_d}$ and $\varepsilon_d = 10^{-6}$ making the distance differentiable at zero, where the Euclidean norm is not.
The hybrid structure is deliberate. The sigmoid saturates at W_{obs} as $d \to 0$: on its own it is a *bounded* penalty, so a sufficiently strong tracking term can buy its way through an obstacle, and — worse for the solver — its gradient vanishes both far away and deep inside the obstacle, leaving a stationary point exactly where the robot must be pushed out. The quadratic term repairs both defects inside the safety radius: it is unbounded and its gradient grows linearly with penetration. The price is that the curvature of (19) scales as $\alpha_{\text{obs}}^2 W_{\text{obs}}$, so a steep, heavy penalty injects large and rapidly varying eigenvalues into the Hessian precisely where the solution lives.
Relaxing a hard constraint into a penalty has three consequences worth stating explicitly. *(i)* The NLP is always feasible: no state constraint can render the problem unsolvable when the robot is already too close to an obstacle, which for a real-time loop with no fallback trajectory is a genuine robustness property. *(ii)* Safety is no longer guaranteed — (19) is not an exact penalty, so a finite W_{obs} admits a finite violation, and Section 3.3.3 makes that statement quantitative. *(iii)* The penalty is evaluated only at the $N + 1$ discrete nodes, so nothing forbids the continuous inter-sample trajectory from clipping a thin obstacle.
Point *(iii)* is usually left as a caveat. It is worth checking instead, because it is a one-line computation: the distance covered between two consecutive nodes is $v_{x,\max}\Delta t$, against a keep-out radius r_{obs}. On the deployed G1 profile that ratio is 0.15 — one step covers 6 cm and 3.4 degrees against a radius of 0.40 m, so the penalty cannot be stepped over. On the Go2 profile the same ratio is 1.02: between two nodes the robot covers slightly more than the entire safety radius, and the penalty is doing nothing at all in between. The Go2 configuration is on the wrong side of its own caveat, and the reason it survives is that the tuning had already demoted the penalty to a last-metre guard, delegating geometric avoidance to the $A^\star$ layer on the inflated grid.

Sentinel padding. M is fixed. When fewer than M returns lie inside the search radius, the missing rows of the obstacle parameter are filled with a sentinel at 10^3 m, whose contribution to (19) is numerically zero. This apparently cosmetic detail is what allows the NLP to be built *once*: the number of terms, and therefore the sparsity pattern of the Jacobian and Hessian, never changes between cycles, so the symbolic factorisation and the warm start remain valid regardless of how cluttered the scene is.

3.3.3 The obstacle as a true constraint: exact ℓ^1 penalty

The alternative to a penalty in the cost is a constraint with a slack. The deployed code implements both, selectable by a parameter: the obstacle becomes

$$d(x_k, p_j) \;\geq\; d_{\text{safe}} - s_{jk}, \qquad s_{jk} \geq 0, \qquad k = 1, \ldots, N, \tag{20}$$

with the slack penalised in the cost either linearly ($\rho_s \sum s$) or quadratically ($\rho_s \sum s^2$). The question this settles is the one Section 3.3.2 could only raise: at what weight, if any, does the relaxation stop being a relaxation.
The exact-penalty theorem answers it. For a non-smooth ℓ^1 penalty there exists a finite threshold — the largest multiplier of the corresponding hard constraint — above which the residual violation is exactly zero, whereas an ℓ^2 penalty decays like $1/\rho_s$ and never vanishes at any finite weight. Both predictions are testable here, because the multiplier is available from the problem itself.
Solved as a hard constraint on a narrow-gap scenario with $d_{\text{safe}} = 1.10$ m — chosen so that the constraint genuinely bites — the instance is feasible and the largest multiplier of an active distance constraint is $\mu^\star = 8905$.

Table 5: Residual constraint violation against the penalty weight, for the non-smooth and the smooth relaxation of the same distance constraint. Bold: the violations that are exactly zero, which is the result the exact-penalty theorem predicts and the smooth relaxation never attains. Zero entries are below the solver tolerance of 1×10^{-8} m.

ρ_s	max slack, ℓ^1 [m]	max slack, ℓ^2 [m]
1×10^3	1.46×10^{-1}	2.61×10^{-1}
1×10^4	**0**	1.30×10^{-1}
1×10^5	**0**	3.98×10^{-2}
1×10^6	**0**	4.38×10^{-3}
1×10^7	**0**	4.42×10^{-4}
1×10^8	**0**	4.42×10^{-5}

The measurement matches the theorem on both branches. The ℓ^1 slack is a threshold phenomenon: nonzero below, and zero to solver tolerance from $\rho_s = 1 \times 10^4$ onwards — which is the first sampled weight above $\mu^\star$.

The ℓ^2 slack decays with a fitted log–log slope of -1.00 against a predicted -1, and is still nonzero five decades later. The practical consequence is that ρ_s stops being a hyperparameter: it is read off the multipliers of the problem rather than tuned.

Two defects of formulation found by implementing it. Both have theoretical content and both are worth reporting, because each is a way to make a well-posed problem unsolvable.
First, **the constraint must not be imposed at** $k = 0$, for the reason already given in Section 2.6: there the state is fixed by an equality, the constraint depends on no decision variable, and imposing it makes the program infeasible exactly when the robot is already inside d_{safe} — that is, exactly when one would want it. Measured on a recorded cycle, the slack saturated at precisely the violation present at $k = 0$ and the solver could do nothing about it.
Second, **the residual infeasibility is a property of $\mathcal{U}$, not of the data**. Even starting from $k = 1$, a demanding d_{safe} remains infeasible on real cycles: with $v_x \geq 0$ and $v_{y,\max} = 0.02$ m/s the robot can only advance along its own heading, and cannot back away laterally from a wall of LiDAR points. This is the interaction between the asymmetry of the admissible input set and a state constraint, and it reappears in Sections 4.5 and 3.3.7 from two other directions.

What it would cost to deploy. Switching the obstacle from the cost to the constraints takes the inequalities from 60 to 300 and the iteration count up by a factor of several. The deployed iteration cap would not suffice. This is also what makes the solver comparison of Section 4.4 possible at all: the same system can be placed in two regimes that differ by a factor five in the number of inequalities, which is exactly the axis along which the standard advice on solver choice is stated.

3.3.4 Constraints

The deployed program carries only equality constraints and input bounds:

$$x_{k+1} - f(x_k, u_k) = 0, \qquad k = 0, \ldots, N-1, \tag{21a}$$
$$x_0 - \hat{x} = 0, \tag{21b}$$
$$0 \;\leq\; u_{1,k} \;\leq\; v_{x,\max}, \qquad k = 0, \ldots, N-1, \tag{21c}$$
$$|u_{2,k}| \leq v_{y,\max}, \qquad |u_{3,k}| \leq \omega_{\max}, \qquad k = 0, \ldots, N-1. \tag{21d}$$

Three remarks.
No state constraints. Neither the obstacle clearance nor the velocity states are constrained; the former is penalised, the latter are bounded implicitly because (14) is a convex combination of a bounded command and the previous velocity. The feasible set is therefore a box intersected with an affine-in-X manifold, and a feasible point is always trivially available.
Forward velocity is non-negative. Constraint (21c) forbids reversing. On the Go2 this was a deployment preference; on the G1 it is a necessity, because the MID-360 covers $-7°$ to $+52°$ in elevation and leaves a blind cone behind the robot, so reversing means moving blind. It is also, as Sections 3.3.3 and 4.5 show, the constraint that most restricts what any state requirement can ask of the robot.
Bounds are parameters, not constants. The three velocity limits enter the NLP as CasADi parameters, so a supervisor can shrink them at runtime without rebuilding the problem.

3.3.5 Terminal ingredients and recursive feasibility

The deployed formulation carries no terminal set and no terminal constraint, so none of the standard guarantees of nominal stability and recursive feasibility [6] applies. Recursive feasibility is nevertheless *structurally* present, for the reason given in Section 3.3.4: with no state constraints, every input sequence in the box is admissible, so the NLP always has a solution and the closed loop can never be aborted by infeasibility. What is lost is the guarantee that the closed loop converges.
That is the situation to improve on, and the deployed code implements the improvement: a terminal equilibrium constraint $v(N) = 0$ — there must exist a braking trajectory inside the horizon — relaxed by a slack and penalised in ℓ^1 norm, so that by the result of Section 3.3.3 the constraint is effectively hard whenever it is satisfiable and yields gracefully when it is not.
Across the cycles replayed from a recorded mission the terminal slack never leaves zero (maximum 0, always admissible: yes), and the cost of imposing it, measured as the relative increase of the optimal objective, ranges from 3.3% to 85.2%. The upper end is paid exactly where the objective was small, that is where the robot was travelling well and now has to predict its own braking — which is the expected trade: a terminal constraint buys the stability argument by spending part of the horizon on stopping.

Why this result must be reported with its cause attached. That the slack is always zero is not, by itself, evidence that the constraint works: an unimplemented constraint would report the same thing. The reason it is zero here is the degeneracy of Section 2.5. With $\nu = 1.000000$ the model reduces to $v_{k+1} = u_k$ and the robot reaches zero velocity in a *single step*, so the terminal set is trivially reachable from anywhere and the constraint costs nothing in admissibility. A counter-test with realistic time constants confirms the mechanism: as τ grows the slack becomes strictly positive and grows with it, because the horizon is no longer long enough to stop. On hardware, with an identified τ, this constraint is the one that would decide the maximum speed from which the robot can still be brought to a halt inside the horizon — which is the physical reading of the terminal feasible set, and the form in which it would actually bind.

3.3.6 Reference trajectory: time-parametrised against path-parametrised

The deployed reference advances along the smoothed $A^\star$ path at a constant cruise speed,

$$s_k = \min(s_0 + v_{\text{ref}}\, k\, \Delta t,\ L_{\text{path}})\,, \tag{22}$$

interpolating the position inside the containing segment and taking the heading as the segment tangent. The cruise speed is set below $v_{x,\max}$ deliberately: with $v_{\text{ref}} = v_{x,\max}$ the tracker saturates permanently and never settles onto the path. The price is structural: 33% of the available forward speed is unreachable *by construction*, and no cost weight can recover it, because the reference never asks for it.
The alternative the theory offers is to make the path abscissa a decision variable: the tracker chooses how far to advance rather than being told. The deployed code implements this too, with $\theta(0) = 0$, $\Delta\theta \geq 0$, $\theta(N) \leq 1$ and a progress term $\alpha_3(1-\theta)^2$ in the cost. The reference $\bar{z}(\theta)$ is represented by a polynomial whose coefficients are parameters, refitted at every solve on the $A^\star$ path — necessary because θ is now a decision variable and IPOPT is a Newton-type method, so a piecewise-linear reference through waypoints would have a discontinuous second derivative at every node.

Table 6: Time-parametrised against path-parametrised reference, averaged over 6 cycles replayed from the recorded run `industrial_plant_v3`.

quantity	time-parametrised	path-parametrised
mean v_x [m/s]	0.189	0.300
advance over the horizon [m]	0.568	0.894
IPOPT iterations	10.7	19.7

Over 6 moving cycles replayed from the recorded run, the mean commanded forward speed rises from 0.189 to 0.300 m/s and the distance covered within one horizon from 0.568 to 0.894 m: the robot advances 58% further at the same horizon, and the solver saturates $v_{x,\max}$ on nearly every step. The cruise-speed parameter does not reappear in another guise — θ is a variable, not a hyperparameter.
The cost is 10.7 → 19.7 IPOPT iterations, an increase of about 70%: the program has $N + 1$ more variables and N more inequalities, and it is less well conditioned. Reported as a trade rather than as an improvement, this is the cleanest reformulation available to the stack.

A trap worth documenting. On cycles where the robot is stationary the comparison is meaningless: θ advances without the robot being able to follow, and with a strong progress weight one observes $\theta \to 1$ while the robot stands still — the reference detaches from the dynamics. The comparison is therefore restricted to cycles with genuine motion. This is not a numerical artefact but a modelling statement: a path-parametrised reference is only a reference if the dynamics can realise the progress it encodes.

3.3.7 Robust constraints: tightening from the measured prediction error

Every constraint in (12) is imposed on the *predicted* trajectory, and Section 5.3 measures how far the prediction drifts from what the robot actually does. A clearance guaranteed in the plan is therefore not a clearance guaranteed on the floor. The standard remedy is constraint tightening: replace d_{safe} with a margin $\gamma(k)$ that grows along the horizon,

$$\left\|p_k - o_j\right\| \ \geq\ d_{\text{safe}} + \gamma(k) - s_{jk}. \tag{23}$$

The particularity of this project is that $\gamma(k)$ need not be postulated. It is read off the 95th percentile of the prediction error recorded in the bags, so the tube is derived from data rather than from an assumption about the disturbance.

Table 7: Constraint back-off derived from the 95th percentile of the measured prediction error. The offset at $k = 0$ is subtracted: it is time misalignment, not model uncertainty, and including it would inflate the tube by a constant.

k	$k\,\Delta t$ [s]	$\gamma(k)$ [m]
0	0.0	0.0000
1	0.2	0.0097
2	0.4	0.0207
3	0.6	0.0524
4	0.8	0.0948
5	1.0	0.1334
6	1.2	0.1658
7	1.4	0.1970
8	1.6	0.2239
9	1.8	0.2495
10	2.0	0.2753
11	2.2	0.3037
12	2.4	0.3353
13	2.6	0.3663
14	2.8	0.3845
15	3.0	0.4146

Three properties hold by construction and were checked rather than assumed. $\gamma(0) = 0.0000$ exactly, because at the first node the state is fixed by an equality and there is nothing to hedge against — the constraint is not tightened where tightening could only remove admissible motion. The margin is monotone (yes), as uncertainty is. And it reaches 0.4146 m at the end of the horizon, the same order as the safety radius itself. The offset at $k = 0$ is subtracted before fitting, since Section 5.3 shows it to be time misalignment rather than model error, and including it would inflate the whole tube by a constant.

Table 8: Effect of the tightening on the predicted trajectory.

scenario	d_{safe}	clear. without	clear. with	Δ	slack w/o / with	outcome
narrow_gap	0.40	0.7500	0.8146	0.0646	0 / 0	tightening effective
narrow_gap	0.70	0.7500	1.1146	0.3646	0 / 0	tightening effective
narrow_gap	1.00	1.0000	1.2816	0.2816	0 / 0.133	infeasible
u_trap	0.40	1.3417	1.3417	0.0000	0 / 0	constraint inactive
u_trap	0.70	1.3417	1.3417	0.0000	0 / 0	constraint inactive
u_trap	1.00	1.3417	1.4146	0.0729	0 / 0	tightening effective

Measured on the predicted trajectory, the outcomes separate cleanly. Where $d_{\text{safe}} + \gamma$ stays below the clearance the trajectory already keeps, the constraint is inactive and correctly does nothing. Where it bites, the predicted clearance increases by up to 0.365 m with the slack still exactly zero: the margin is *respected*, not violated and paid for — which is the whole point of pairing the tube with the ℓ^1 relaxation of Section 3.3.3, since a tube that asks more than $\mathcal{U}$ can deliver makes the penalty yield instead of making the program infeasible.

The limit of this measurement, and what it says about the architecture. The effect is *not* observable in the closed loop of this harness: the executed clearance comes out identical with and without tightening. The reason is the architecture described in Section 2.1 — the loop is closed by tracking a look-ahead setpoint on the predicted trajectory with a proportional law, which washes out fine differences between MPC solutions. The tightening guarantees the margin *in the plan*, and that is where it has been verified. This is not a limitation of the technique but of the test bench, and it is the strongest argument in the report for applying $u_0^\star$ directly, as the receding-horizon principle prescribes.

On the recorded industrial run the tightening is inert for a different and more instructive reason: at the tightest cycles the robot sits about 0.6 m from the obstacles and cannot move away. Raising d_{safe} grows the slack but does not move the trajectory. The binding constraint is not the threshold, it is $\mathcal{U}$ — the same asymmetry found in Section 3.3.3 and quantified in Section 4.5. Three independent measurements arrive at the same conclusion, which is the strongest form the report has for saying that the input set, not the cost, is where this system is limited.

4. Optimization procedure

4.1. Problem dimensions and structure

Table 9 reports the size of the NLP against the horizon, measured on the deployed profile rather than counted by hand.

Table 9: Size and sparsity of the NLP against the prediction horizon, from the deployed profile. Bold: the deployed configuration.

N	vars	eq.	bounds	params	jac nnz	jac dens. [%]	hess nnz	hess dens. [%]
10	96	66	40	91	256	2.52	600	6.51
15	**141**	**96**	**60**	**121**	**381**	**1.73**	**885**	**4.45**
25	231	156	100	181	631	1.07	1455	2.73
50	456	306	200	331	1256	0.54	2880	1.39

At the deployed $N = 15$ the program carries 141 decision variables and 156 constraints, of which 96 are equalities — the dynamics plus the initial condition — and 60 are simple bounds on the inputs. A further 121 quantities enter as CasADi parameters: the measured state, the reference, the sentinel-padded obstacle positions and the three velocity bounds. The problem is therefore medium-scale, sparse, smooth and non-convex.

The constraint Jacobian is 1.73% dense and the Hessian of the Lagrangian 4.45%. Both densities fall as N grows while the nonzero counts grow linearly, which is the signature of the simultaneous parametrisation: the Jacobian of (21a) is block bi-diagonal, the Hessian block tri-diagonal with the off-diagonal input blocks coming from the rate penalty, and nothing couples distant stages. The linear-algebra cost of one interior-point iteration is consequently linear in N.

It is worth being explicit about what is *not* in this table, because it shapes everything in Section 4.4. There are no obstacle constraints: the $2M(N+1)$ obstacle terms all live in the objective. There is therefore no obstacle multiplier, no non-trivial active set to identify, and the entire inequality budget of the deployed program is 60 box bounds on the inputs.

4.2. Derivative computation

Every quantity the solver needs — the gradient of the objective, the Jacobian of the constraints, the Hessian of the Lagrangian — is generated symbolically by CasADi and evaluated by generated code. The alternative is finite differences, and the comparison is not a formality: it decides whether the second-order method of Section 4.3 is affordable at all.

Table 10: Accuracy of the finite-difference gradient against the step size, measured against the AD gradient. The optimum sits at $h = 1.49\times10^{-8}$ one-sided (theory: $\sqrt{\varepsilon} = 1.49\times10^{-8}$) and $h = 6.06\times10^{-6}$ central (theory: $\varepsilon^{1/3} = 6.06 \times 10^{-6}$).

step h	rel. error, one-sided	rel. error, central
1.00×10^{-4}	2.48×10^{-4}	2.80×10^{-8}
6.06×10^{-6}	1.50×10^{-5}	1.29×10^{-10}
1.00×10^{-6}	2.48×10^{-6}	5.43×10^{-10}
1.49×10^{-8}	5.50×10^{-8}	3.45×10^{-8}
1.00×10^{-10}	8.03×10^{-6}	4.31×10^{-6}

At a representative solve the objective has 141 variables. One evaluation costs 105.0 μs and one full gradient by reverse-mode algorithmic differentiation (AD) 145.4 μs, i.e. 1.51 objective evaluations — a small constant, and crucially one that does not grow with the number of variables. The same gradient by finite differences costs 142 evaluations one-sided and 282 central, both growing linearly with n, and is less accurate at every step size: the best relative error attainable is 5.50×10^{-8} one-sided and 1.29×10^{-10} central, against machine precision for AD. The measured optimal steps coincide with the theoretical $\sqrt{\varepsilon_M}$ and $\varepsilon_M^{1/3}$, ε_M being the machine epsilon.

For a real-time loop the cost settles the question: central differences alone would consume 24% of the 125 ms cycle budget, for a gradient that is worse, and the fraction grows linearly with the horizon. The ratio itself is a

micro-benchmark on $\sim$100 μs timings and its second digit is not meaningful; what is stable, and is the point, is that it remains a small constant.

The exact Hessian, and what it buys. Because AD supplies exact second derivatives at a cost comparable to the first, the full Newton system can be used rather than a quasi-Newton approximation.

Table 11: Interior-point iterations with the exact Hessian and with the limited-memory quasi-Newton approximation, on the same instance. Bold: the lower iteration count.

Hessian	iterations	solve [ms]	$J^\star$	status
exact Hessian	**20**	210.9	9244	`Solve_Succeeded`
L-BFGS	36	297.1	9244	`Solve_Succeeded`

On the same instance the exact Hessian converges in 20 iterations against 36 for the limited-memory quasi-Newton approximation — a 44% reduction — and both reach the same objective value. The total time is closer than the iteration counts suggest, because each quasi-Newton iteration is cheaper: this is the Newton / quasi-Newton trade-off in its textbook form, measured without writing a solver. Section 4.4 shows the other side of the same coin, where the exact Hessian of a non-convex program produces an indefinite quadratic program (QP) subproblem.

4.3. Solver methodology

The NLP is solved with IPOPT [11], a primal–dual interior-point method with a filter line search. Writing the problem as $\min_z \phi(z)$ subject to $c(z) = 0$ and $z^L \le z \le z^U$, the bounds are replaced by a logarithmic barrier

$$\varphi_{\mu_b}(z) \;=\; \phi(z) \;-\; \mu_b \sum_i \Big[\ln\big(z_i - z_i^L\big) + \ln\big(z_i^U - z_i\big) \Big], \tag{24}$$

the resulting equality-constrained problem is solved approximately by a Newton step on the perturbed Karush–Kuhn–Tucker (KKT) system

$$\begin{bmatrix} \nabla^2_{zz}\mathcal{L} + \Sigma & \nabla c(z)^\top \\ \nabla c(z) & 0 \end{bmatrix} \begin{bmatrix} \Delta z \\ \Delta\lambda \end{bmatrix} = - \begin{bmatrix} \nabla\varphi_{\mu_b} \\ c(z) \end{bmatrix}, \tag{25}$$

and μ_b is driven to zero. Each iteration requires one factorisation of the sparse symmetric indefinite matrix in (25), performed by MUMPS on the pattern of Section 4.1.

There is a pleasing coincidence between (24) and (19) that is worth naming: the obstacle term of the deployed program is itself a barrier, and α_{obs} plays the role of an inverse barrier parameter. Steeper means better avoidance and worse conditioning, which is exactly the trade-off μ_b governs inside the solver. The difference is that IPOPT drives its own barrier parameter to zero along a central path, whereas α_{obs} is fixed by the tuning — so the program is solved with a barrier that never relaxes.

4.4. Interior point against active set

The standard advice is that active-set methods win when the inequalities are few and interior-point methods win when they are many. This stack can be moved between the two regimes without changing anything else, because the obstacle formulation is switchable: with the obstacles in the cost the program carries 60 inequalities, and with the obstacles as genuine constraints (Section 3.3.3) it carries 300. That makes the advice testable rather than quotable.

The rule does not reproduce: the active-set solver does not win even in the small regime, where the interior-point method is already 33.5× faster. The *direction* is confirmed — the margin widens to 10.0× when the inequalities multiply — so the break-even point, if it exists, lies below the smallest regime into which this problem can be put.

Two cautions, without which the numbers would be misleading. First, the real advantage of an active set in MPC is warm starting *between consecutive solves*: the active set changes by a few rows per cycle and the factorisation is reused, which is the online active-set strategy the method was designed for. Both solvers are started cold here, deliberately, so as to favour neither — which removes from the active-set method precisely what makes it competitive in a receding-horizon loop. Second, the advice is stated for convex programs, and this one is not.

Table 12: The same instance solved by a primal–dual interior-point method and by an SQP with an active-set QP solver, in both obstacle regimes. Both are started cold. Bold: the faster of the two.

regime	ineq.	IPOPT ms / iter	SQP ms / iter	speed-up	same min.
penalty (obstacles in the cost)	60	**49** / 21	1656 / -1	33.5×	no
penalty (obstacles in the cost)	60	**34** / 15	138 / 6	4.0×	yes
penalty (obstacles in the cost)	60	**47** / 16	348 / 6	7.5×	yes
ℓ^1 (obstacles as constraints)	300	**76** / 21	5216 / 11	68.7×	no
ℓ^1 (obstacles as constraints)	300	**112** / 21	5000 / 7	44.7×	yes
ℓ^1 (obstacles as constraints)	300	**45** / 12	453 / 1	10.0×	no

A by-product that the theory predicts. With the exact Hessian of the Lagrangian, the QP subproblem of the sequential quadratic programming (SQP) method is repeatedly flagged indefinite. This is expected and instructive: a non-convex program can produce a non-convex QP, and a non-convex QP has no unique solution. It is exactly the reason a Gauss–Newton Hessian — positive semi-definite by construction — is the standard recommendation for SQP, and it is the counterpart of the result in Section 4.2, where the same exact Hessian was the better choice inside an interior-point method. The two are not in contradiction: an interior-point method regularises the indefinite system in (25), an SQP must solve the QP.

A methodological note. On a first run the SQP appeared to converge in a single iteration to a worse objective than IPOPT. Comparing the time of two solves that terminate at *different* minima means nothing, and the tool now verifies that the two objectives agree before declaring a winner. Table 12 reports that check in its last column.

4.5. Optimality conditions on the deployed problem

Everything above assumes that what the solver returns is a solution. That is a verifiable statement, and this section verifies it on cycles extracted from a recorded mission rather than on a synthetic instance.

Table 13: Optimality diagnostics at sampled cycles of the recorded run `industrial_plant_v3`. "active" counts equalities and active bounds together; "rank" is the rank of the Jacobian of the active constraints, so LICQ holds when the two coincide.

cycle	t [s]	active	of which bounds	rank	LICQ	cone dim.	$\lambda_{\min}$ proj.
0	0	96	0	96	yes	45	8.78×10^{-1}
95	60	98	2	98	yes	43	8.75×10^{-1}
190	105	100	4	100	yes	41	8.37×10^{-1}
285	153	97	1	97	yes	44	8.78×10^{-1}
380	197	96	0	96	yes	45	8.78×10^{-1}
475	242	105	9	105	yes	36	9.16×10^{-1}
570	294	99	3	99	yes	42	8.83×10^{-1}
665	359	137	41	137	yes	4	7.19×10^{0}
761	430	136	40	136	yes	5	1.57×10^{0}

At 9 sampled cycles, the linear independence constraint qualification (LICQ) holds at every one (yes) — the rank of the Jacobian of the active constraints equals their number — so the KKT multipliers exist and are *unique*. Strict complementarity holds at every one (yes), which means the critical cone coincides with the null space of the active Jacobian and the second-order condition can be checked exactly rather than conservatively. And the reduced Hessian is positive definite on that cone at every cycle (yes), the smallest projected eigenvalue over the profile being 8.37×10^{-1}. Taken together these are a certificate of strict local optimality, which is the most that can be claimed for a non-convex program.

The result worth reporting is the shape of the active set. The critical cone contracts from 45 dimensions at the first sampled cycle to 5 at the last: with 141 variables and 136 active constraints, almost no freedom remains. The controller moves from *cost-driven* to *constraint-driven* over the course of the mission, and towards the end it does not so much choose the trajectory as undergo it. The breakdown of the active

bounds explains it: the lateral and yaw limits are saturated on essentially every step of the horizon, and the non-negativity of v_x on nearly all of them.
This is the quantitative form of the statement made three times elsewhere in this report. A lateral bound of $v_{y,\max} = 0.02$ m/s does not *limit* a degree of freedom, it *removes* it. Section 3.3.3 found the residual infeasibility of the tightened obstacle constraint to be a property of $\mathcal{U}$; Section 3.3.7 found the tube inert on the real run for the same reason; and here the same fact appears as a critical cone of dimension 5. No choice of cost weights can return what the input set has taken away.

Two traps in reading multipliers out of a modelling layer. Both were met here and both would have produced quietly wrong numbers. CasADi's `Opti` canonicalises every constraint as $\text{lbg} \le g(z) \le \text{ubg}$ and *absorbs into the bounds* any parametric right-hand side, so $X_{:,0} = \hat{x}$ becomes $g = X_{:,0}$ with $\text{lbg} = \text{ubg} = \hat{x}$: evaluating $|g|$ on those rows returns the state, not the residual, which is always $g - \text{lbg}$. And two distinct rows can share the same expression with different bounds — $u_{1,k} \ge 0$ and $u_{1,k} \le v_{x,\max}$ are both the row $u_{1,k}$ — so activity must be decided on the distance from the bound, not on $|g| \approx 0$, or both sides of every box are counted active.

4.6. Implementation and configuration

The program is built once, symbolically, and every quantity that changes between cycles is a parameter rather than a constant: the measured state, the reference, the sentinel-padded obstacle positions and the three velocity bounds. Consequently the expression graph, the derivative code and the sparsity patterns are generated exactly once at start-up, and a cycle only writes numbers into parameter slots and calls the solver. The graph is flattened so that the interpreter overhead of a problem with thousands of elementary nonlinear operations does not dominate. The first solve after start-up therefore pays a one-off construction cost, which is the single deadline violation of an entire run and is absorbed by holding the robot still during initialisation.
The solver options are deliberately austere, with a low iteration cap: a non-converged iterate is still usable as a reference by the downstream setpoint stage, which is a property of the architecture of Section 2.1 and would not hold if $u_0^\star$ were applied directly. The real-time protection lives in the supervisor rather than inside IPOPT — a solver whose behaviour depends on a wall-clock limit is not deterministic and cannot be characterised. The supervisor clears the warm-start cache on solver failure, falls back to a zero command after repeated failures, and can shrink the velocity bounds through the parameter interface when the failure rate rises.

5. Performance analysis

5.1. Experimental setup

All results in this section are produced by an offline harness that imports the *production* modules — the grid, the $A^\star$ planner and the MPC tracker — so every number is generated by the deployed formulation and the deployed solver, not by a re-implementation. Two kinds of experiment are used and they answer different questions.
Replay from recorded missions. The bag `industrial_plant_v3` contains a full run of the G1 in the industrial world: the poses, the LiDAR scans, the $A^\star$ paths and the MPC predictions as they were published. Replaying it gives real operating points, with the geometry and the clutter the robot actually met, and is what Sections 5.3 and 4.5 use. It cannot answer counterfactual questions.
Closed-loop synthetic scenarios. For counterfactuals — what if the horizon were shorter, what if the weights were different — the harness runs full missions in scripted worlds, with a ray-marched planar LiDAR and the discrete model of Section 3.2 as the plant. This is what Sections 5.5, 5.6 and 5.7 use. Two of these worlds recur: a narrow gap and a U-shaped trap.
A caution about the second family, which the report would be weaker for omitting: in the U-trap the minimum clearance comes out at zero in essentially every configuration tested. The robot grazes an obstacle regardless of the horizon, the control horizon or the weights. That world therefore discriminates on time and on computation but says nothing about safety, and any comparison of clearances drawn from it would be comparing two configurations that both touch.

5.2. The cost landscape, and why this is not a potential field

Before the campaigns, it is worth looking at the object all of them are about. Fig. 1 draws, over the world plane of the recorded industrial run, the cost of *being* at a point,

$$c(p) \;=\; \underbrace{\text{tracking cost of the reference}}_{\text{the } Q \text{ block of Eq. (18)}} \;+\; \underbrace{J_{\text{obs}}(p)}_{\text{Eq. (19)}}, \tag{26}$$

with the executed trajectory, the $A^\star$ reference and one MPC horizon drawn on the surface, and the per-cycle optimal cost $J^\star$ underneath.

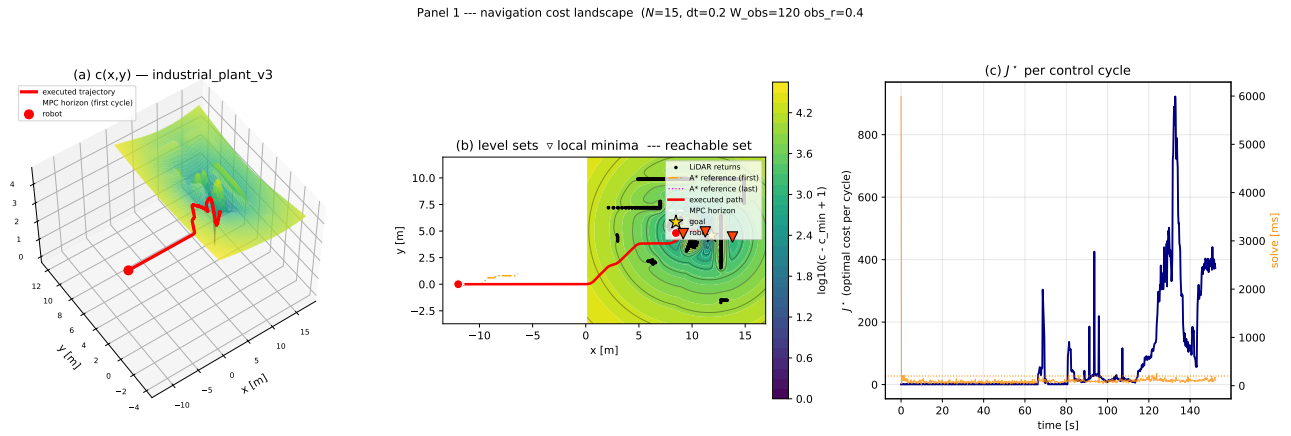

Figure 1: The navigation cost landscape of the recorded run `industrial_plant_v3`. **(a)** the surface $c(p)$ of Eq. (26) over the world plane, with the executed trajectory (red) and one MPC horizon (dashed) drawn *on* the surface: the obstacle term raises a ridge along every wall of the plant, the tracking term forms the valley along the $A^\star$ reference. **(b)** the same field as level sets, with the local minima marked and the shaded region showing the set reachable within one horizon under $\mathcal{U}$ — the constraint, not the cost, is what excludes most of the plane. **(c)** the optimal cost $J^\star$ of each control cycle, and the solve time beside it. The point worth noticing is in panel (a): *the trajectory does not follow steepest descent*, and climbs the surface wherever doing so lowers the sum over the horizon.

The distinction that figure makes visible is the one between c and J, and it is the reason the stack uses an optimizer rather than a reactive law. The quantity plotted is c, the cost of occupying a point; what the program minimises is

$$J(U) \;=\; \sum_{k=0}^{N} c(p_k) \;+\; \text{input terms}, \tag{27}$$

the cost of a whole *trajectory*. A controller that descended c pointwise would be an artificial potential field, and it would stop at the first local minimum of the surface. The MPC minimises the sum along a horizon instead, so it can — and in panel (a) visibly does — move *uphill* on c when that buys a lower total. This is what lets the same cost function carry the robot out of a concave pocket, and it is the sense in which the obstacle term of Eq. (19) is a penalty inside an optimization problem rather than a repulsive force.

One further reading, which anticipates Section 4.5. The shaded region in panel (b) is not a level set of the cost: it is the set of points the robot can reach within one horizon given $\mathcal{U}$. Most of the plane is excluded from the decision by the *constraints*, not by the objective — and on this platform, with reversing forbidden and $v_{y,\max} = 0.02$ m/s, that region is markedly asymmetric about the heading. The cost is what chooses inside it; the input set is what draws it.

5.3. Prediction error: the model against the plant

The most consequential measurement in this report requires no new experiment: the bags already contain, at each cycle, the trajectory the MPC predicted, and the poses the robot subsequently reached. Comparing the prediction published at time t with the pose measured at $t + k\Delta t$, over 762 usable cycles, gives the growth of the prediction error along the horizon.

The residual at $k = 0$ is 0.0241 m and is *not* model error: at the first node the predicted state *is* $\hat{x}$, imposed as an equality constraint. It measures the delay between the instant the prediction is published and the instant

Table 14: Open-loop prediction error along the horizon, over 762 cycles of the recorded run `industrial_plant_v3`.

k	$k\,\Delta t$ [s]	median error [m]	95th pct. [m]
0	0.0	0.0241	0.0610
1	0.2	0.0295	0.0707
2	0.4	0.0443	0.0817
3	0.6	0.0425	0.1134
4	0.8	0.0529	0.1557
5	1.0	0.0754	0.1943
6	1.2	0.1032	0.2267
7	1.4	0.1273	0.2580
8	1.6	0.1521	0.2849
9	1.8	0.1762	0.3105
10	2.0	0.2027	0.3363
11	2.2	0.2302	0.3647
12	2.4	0.2561	0.3963
13	2.6	0.2819	0.4273
14	2.8	0.3064	0.4455
15	3.0	0.3291	0.4756

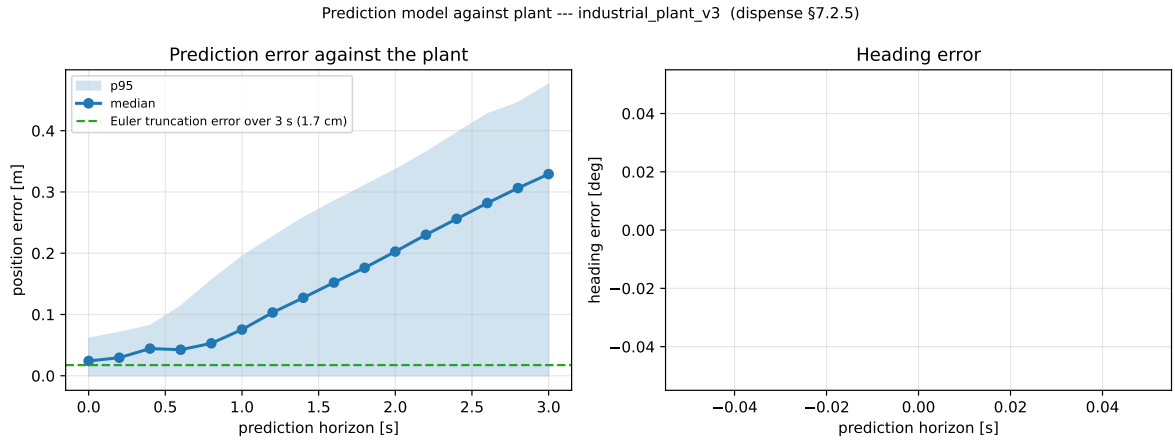

Figure 2: Prediction error against the position along the horizon, over 762 cycles of the recorded run `industrial_plant_v3`: median and 95th percentile. The residual at $k = 0$ is time misalignment between the instant the prediction is published and the instant the pose is sampled, not model error, and is subtracted before quoting the divergence.

the pose is sampled, and it is subtracted to isolate the genuine divergence, which is 0.305 m at the end of the horizon.
Put beside Section 3.2, that number closes a circle and is uncomfortable. The divergence is 18 times the truncation error of forward Euler over the same window, and 3505 times that of the mid-point rule now deployed. The integrator was never the dominant term. Upgrading it improved the arithmetic of the prediction by a factor 200 and left the closed loop essentially unchanged, and this is why: what limits the prediction is not how the model is integrated but that the model is a planar holonomic agent and the plant is a twenty-nine-degree-of-freedom humanoid walking. The correct response is not a better integrator but either a better model or, more cheaply, an explicit account of the residual — which is what the constraint tightening of Section 3.3.7 does with this very curve.

5.4. Regularity of the solution and the left–right bifurcation

The program is non-convex, so its solution need not depend continuously on the data. The place where this is visible is an obstacle centred on the reference: the problem then admits two symmetric local minima — pass left, pass right — and the MPC law is a well-defined function of the state only as long as one of them dominates. The experiment solves the *same* instance twice, from a left-biased and a right-biased initial guess, and measures the distance between the two returned trajectories as the obstacle weight is swept.

Table 15: Left-biased against right-biased solve of the same instance, against the barrier weight. The separation is the distance between the two returned trajectories: a value at solver tolerance means the two guesses converged to the same minimum.

W_{obs}	separation [m]	$J^\star$ left	$J^\star$ right	iter. left	iter. right
60.0	1.16×10^{-6}	4953	4953	21	21
120	6.34×10^{-9}	9244	9244	19	19
200	7.11×10^{-10}	14252	14252	24	23
300	3.69×10^{0}	20013	20122	25	29
450	3.89×10^{0}	28152	28297	25	23
600	3.98×10^{0}	36019	36187	27	28
900	4.22×10^{0}	51333	51596	29	25
1400	4.49×10^{0}	76236	76606	24	26

The transition sits between $W_{\text{obs}} = 200$ and 300, and the deployed weight (120) is below it: at the tuning in use the minimiser is unique and regular in x_0. Two further readings follow. Past the transition the two minima are *not* equivalent — their objective values differ by up to 0.55% — so the initial guess decides not only which side the robot passes but how much the manoeuvre costs. And on the hardest cycle of the recorded run, cycle 664, the sweep never bifurcates (yes) even far above the threshold, which Section 4.5 explains: that cycle has a critical cone of dimension one, and with a single remaining degree of freedom there is no room for two distinct basins. Constraint saturation kills the bifurcation before the weight can create it.
A practical consequence for the deployed code: the guard that clears the warm-start cache when the objective jumps between cycles is protecting against a phenomenon that does not occur at these weights.

Seeing it in the decision space. The sweep above measures the bifurcation; Fig. 4 shows it. Plotting a function of 141 variables requires choosing a two-dimensional section, and a random one would almost surely contain nothing: the plane here is built to contain the structure. The program is solved twice, warm-started to the left and to the right, giving the two minima $z_L^\star$ and $z_R^\star$; the initial iterate supplies a third anchor; and the affine plane through those three points contains both minima *by construction*. The IPOPT iterates then project onto it exactly, since orthogonal projection onto an affine subspace is exact — which is why this figure can show the path from z^0 to $z^\star$ and Fig. 1 cannot.
The choice of what to plot is itself a point about the formulation. Under multiple shooting the states are decision variables, so almost every point of the plane violates Eq. (21a); the objective is defined there but meaningless, because no trajectory of the plant corresponds to it. The ℓ^1 merit function — objective plus weighted constraint violation — is defined everywhere, agrees with the objective on the feasible manifold, and is the function whose decrease the line search enforces. Plotting it is the honest choice, and it also makes the figure legible: the funnel visible in panel (b) is the constraint violation being driven to zero, and the iterates run down it before moving along the bottom towards the minimum. That two-phase shape is the geometry of a filter line-search method, and it is the same mechanism that Section 3.3.3 exploits when it makes the obstacle constraint exact.

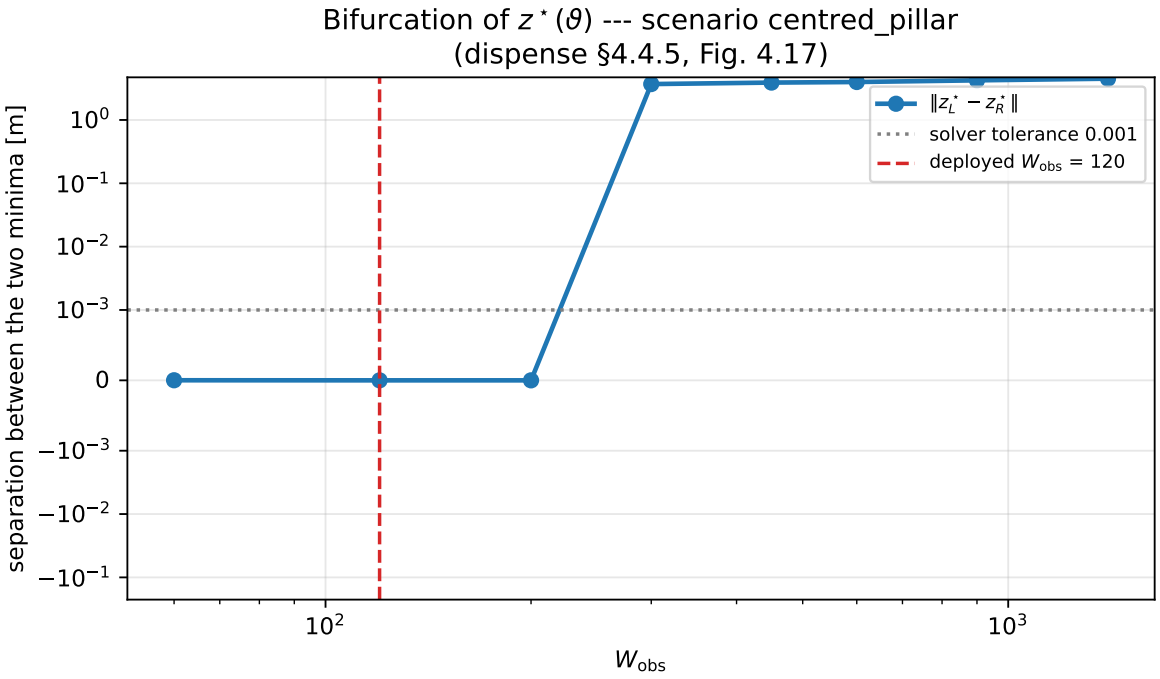

Figure 3: Separation between the trajectories returned from a left-biased and a right-biased initial guess, against the obstacle weight W_{obs}. Below the transition the two guesses collapse onto the same minimum and the control law is a regular function of the state; above it they do not.

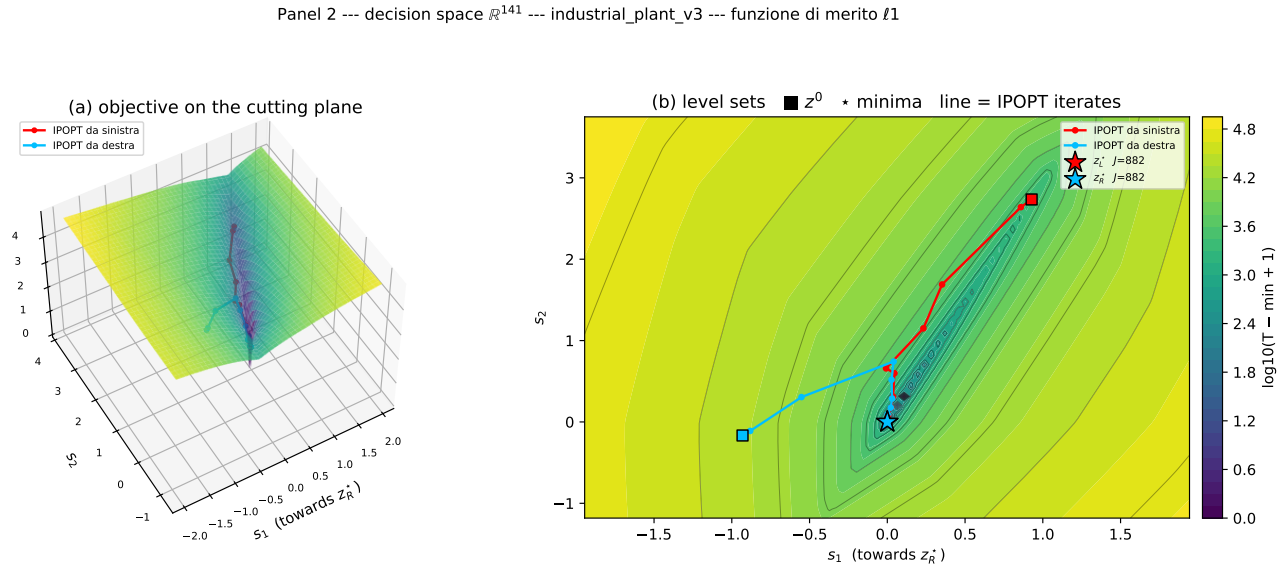

Figure 4: The objective in the decision space $\mathbb{R}^{141}$, on a plane constructed to contain both local minima of the recorded cycle. **(a)** the surface; **(b)** its level sets with the two minima ($\star$), the initial iterate (■) and the IPOPT iterates joining them. What is plotted is not the objective f but the ℓ^1 merit function $T_1(z) = f(z) + \sigma\|c(z)\|_1$: in the simultaneous parametrisation X and U are independent variables tied by the dynamic equalities, so a generic point of the plane is *not* dynamically feasible and a plot of f alone would show a landscape the solver never traverses. The merit function is what the line search actually decreases.

5.5. Prediction horizon and sampling time

The two horizon parameters are not interchangeable. The product $N\Delta t$ sets how far the controller sees, N alone sets how much the solve costs, and Δt alone sets how faithfully the prediction is integrated. They were therefore swept jointly, in closed loop, over 40 configurations.

Table 16: Horizon campaign, scenario `narrow_gap`. Bold: the deployed configuration $N = 15$, $\Delta t = 0.20$ s.

N	Δt [s]	T [s]	vars	median [ms]	p95 [ms]	goal	TTG [s]	min clear. [m]
5	0.10	0.5	51	11.2	26.1	yes	9.1	0.450
5	0.20	1.0	51	9.3	24.7	yes	9.4	0.450
5	0.30	1.5	51	10.9	27.3	yes	9.3	0.451
5	0.40	2.0	51	10.5	22.7	yes	9.6	0.450
10	0.10	1.0	96	15.9	27.6	yes	9.1	0.450
10	0.20	2.0	96	16.9	47.6	yes	9.4	0.450
10	0.30	3.0	96	15.4	45.9	yes	9.3	0.451
10	0.40	4.0	96	17.5	47.7	yes	9.6	0.450
15	0.10	1.5	141	19.8	67.8	yes	9.1	0.450
15	**0.20**	**3.0**	**141**	**20.7**	**60.3**	**yes**	**9.4**	**0.450**
15	0.30	4.5	141	24.3	64.9	yes	9.3	0.451
15	0.40	6.0	141	27.5	95.3	yes	21.6	0.017
25	0.10	2.5	231	38.7	129.2	yes	9.1	0.450
25	0.20	5.0	231	39.4	65.1	yes	9.2	0.450
25	0.30	7.5	231	46.1	78.1	no	—	0.044
25	0.40	10.0	231	37.1	65.1	yes	23.2	0.305
40	0.10	4.0	366	42.0	173.1	yes	9.1	0.450
40	0.20	8.0	366	50.2	90.3	yes	24.2	0.214
40	0.30	12.0	366	61.4	111.0	yes	26.7	0.232
40	0.40	16.0	366	60.0	86.7	yes	23.2	0.296

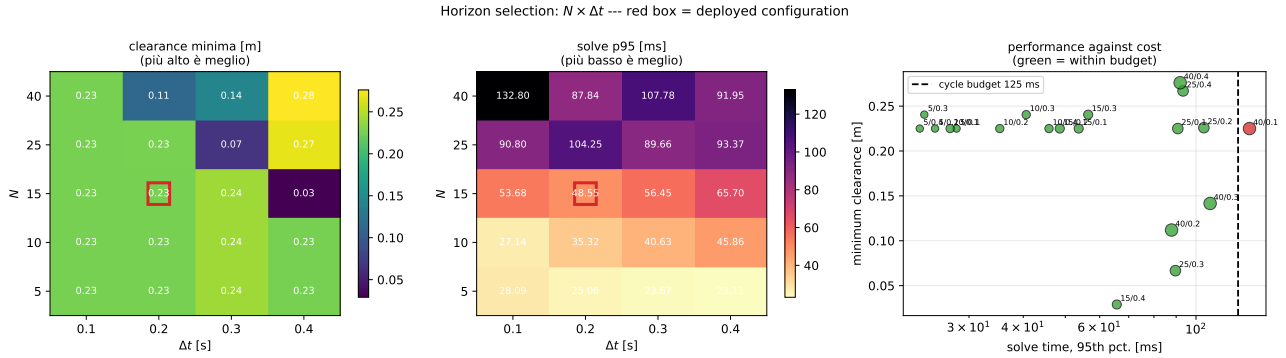

Figure 5: Joint sweep of the prediction horizon N and the sampling time Δt in closed loop. The deployed configuration is marked. Cost is governed by N; closed-loop quality is governed by the product $N\Delta t$, and not monotonically.

The headline is counter-intuitive: beyond roughly 6 s of look-ahead, *lengthening the horizon makes the closed loop worse*. Grouped at that threshold, the short-horizon configurations reach the goal in 11.0 s against 22.7 s for the long-horizon ones. The clearance axis does not discriminate here, for the reason given in Section 5.1 — both groups graze — so only the time is informative, and on time the verdict is unambiguous.

The mechanism is the same one that produces the failure of Section 5.8. The reference extends over a path that the discrete layer will replan anyway, so a longer horizon commits the controller to tracking a target already due to change. Section 5.6 separates this explanation from its competitor — simply too many decision variables — and reports which one the data supports.

Ranking the configurations on the three objectives that matter (time to goal, worst-case clearance, tail solve time) leaves 6 non-dominated points, and the deployed $N = 15$, $\Delta t = 0.20$ s is *dominated*: $N = 5$, $\Delta t = 0.40$ s matches it on task performance at 23.6 ms of tail solve time against 60.3 ms.

Table 17: Horizon campaign, scenario `u_trap`. Bold: the deployed configuration $N = 15$, $\Delta t = 0.20$ s.

N	Δt [s]	T [s]	vars	median [ms]	p95 [ms]	goal	TTG [s]	min clear. [m]
5	0.10	0.5	51	11.9	30.1	yes	12.4	0.000
5	0.20	1.0	51	12.3	25.4	yes	12.8	0.000
5	0.30	1.5	51	11.2	20.1	yes	12.6	0.030
5	0.40	2.0	51	13.1	23.6	yes	12.8	0.000
10	0.10	1.0	96	13.2	26.7	yes	12.4	0.000
10	0.20	2.0	96	14.5	23.0	yes	12.8	0.000
10	0.30	3.0	96	15.9	35.3	yes	12.6	0.030
10	0.40	4.0	96	17.0	44.0	yes	12.8	0.000
15	0.10	1.5	141	18.5	39.5	yes	12.4	0.000
15	**0.20**	**3.0**	**141**	**19.2**	**36.8**	**yes**	**12.8**	**0.000**
15	0.30	4.5	141	21.7	48.0	yes	12.6	0.030
15	0.40	6.0	141	22.5	36.1	yes	18.8	0.041
25	0.10	2.5	231	28.1	52.4	yes	12.4	0.000
25	0.20	5.0	231	33.9	143.4	yes	13.2	0.002
25	0.30	7.5	231	37.4	101.2	yes	18.6	0.089
25	0.40	10.0	231	56.2	121.6	yes	19.2	0.229
40	0.10	4.0	366	42.9	92.5	yes	12.4	0.000
40	0.20	8.0	366	45.8	85.4	yes	26.4	0.009
40	0.30	12.0	366	59.0	104.5	yes	23.7	0.051
40	0.40	16.0	366	44.1	97.2	yes	20.4	0.256

This is the most actionable result in the report, and also the one to act on most carefully. These are synthetic scenarios with static obstacles and frequent replanning; a very short horizon holds up only because the discrete layer is doing the avoidance, and with dynamic obstacles or a slower planner that margin would disappear. The result is a reason to run the comparison on real missions, not a reason to retune from a table.

5.6. Control horizon, and why not move blocking

Section 5.5 leaves a diagnostic question open, because there N governs two things at once: how far the controller looks, and how many free inputs it has. Freeing only the first N_c inputs and holding the rest at the last free value separates them — the prediction still runs to N, but the program carries fewer variables.

Table 18: Closed-loop outcome against the control horizon at fixed prediction horizon.

scenario	N	N_c	vars	goal	TTG [s]	min clear. [m]	p95 [ms]
narrow_gap	5	1	39	yes	9.4	0.450	21.8
narrow_gap	5	5	51	yes	9.4	0.450	12.2
narrow_gap	15	1	99	yes	9.4	0.450	18.7
narrow_gap	15	5	111	yes	9.4	0.450	48.6
narrow_gap	15	15	141	yes	9.4	0.450	40.2
u_trap	5	1	39	yes	12.8	0.000	6.3
u_trap	5	5	51	yes	12.8	0.000	11.4
u_trap	15	1	99	yes	12.8	0.000	11.2
u_trap	15	5	111	yes	12.8	0.000	25.7
u_trap	15	15	141	yes	12.8	0.000	23.8

The result splits in two. Where the prediction horizon is already the right length, cutting the degrees of freedom is *free*: in 3 of the paired cases the time to goal and the minimum clearance are unchanged while the tail solve time falls, by up to 2.1× at $N = 15$. This is the input-parametrisation argument in its cheapest form — the degrees of freedom past the first few were not what the closed loop was using.
The complementary half is the answer to the diagnostic question, and on the present sweep it must be reported as unanswered: none of the horizons tested here degrades the closed loop, so there is no degradation to attribute to either cause. Re-running this comparison over the horizons that *do* degrade is the cheapest missing measurement in this report.

On move blocking. Holding the input constant over blocks of increasing length is the standard way to recover computation without shortening the horizon, and it is deliberately not used here. Wherever Section 5.5 finds the useful horizon to be short, compressing a handful of variables into blocks changes nothing measurable; the control horizon above is the degenerate case of move blocking — one free block followed by one long one — and already delivers the saving; and computation is not the binding resource, since the deployed configuration sits well inside its cycle budget while the prediction error of Section 5.3 does not. It is a considered choice, not an omission, and it would become the right technique if a rigorous terminal ingredient forced a long horizon.

One implementation detail that carries theory. Imposing the input bounds on all N steps when the inputs past N_c are the same repeated expression generates duplicate rows with identical gradients. If active, they violate LICQ and make the multipliers non-unique — breaking exactly the analysis of Section 4.5. The bounds must be imposed on the free columns only, and with that correction LICQ and strict complementarity hold for every N_c.

5.7. Multi-objective scalarisation

The cost (18) is a scalarisation: several competing objectives combined into one stage cost by a convex combination of weights,

$$\min_{U,X} \sum_j \kappa_j f_j(X,U), \qquad 0 \le \kappa_j \le 1, \qquad \sum_j \kappa_j = 1. \tag{28}$$

The three objectives that (18) introduces are tracking accuracy (the Q block), control effort (the R block) and progress along the path (the term in $(1-\theta)^2$ of Section 3.3.6). The weights are scaled so that the barycentre of the simplex reproduces the tuning in use.

Table 19: Closed-loop outcome of each scalarisation of the three objectives. Bold: non-dominated points.

κ	accuracy [m]	effort	time [s]	clearance [m]	non-dom.
(0.0, 0.0, 1.0)	**0.0203**	**0.0904**	**9.0**	**0.450**	yes
(0.0, 0.2, 0.8)	0.0208	0.0870	9.4	0.450	no
(0.0, 0.4, 0.6)	0.0214	0.0870	9.4	0.450	no
(0.0, 0.6, 0.4)	0.0215	0.0870	9.4	0.450	no
(0.0, 0.8, 0.2)	0.0217	0.0870	9.4	0.450	no
(0.0, 1.0, 0.0)	0.0217	0.0870	9.4	0.450	no
(0.2, 0.0, 0.8)	0.0190	0.0865	9.4	0.450	no
(0.2, 0.2, 0.6)	0.0293	0.0930	9.2	0.450	no
(0.2, 0.4, 0.4)	0.0197	0.0867	9.4	0.450	no
(0.2, 0.6, 0.2)	0.0197	0.0867	9.4	0.450	no
(0.2, 0.8, 0.0)	0.0190	0.0865	9.4	0.450	no
(0.4, 0.0, 0.6)	0.0197	0.0867	9.4	0.450	no
(0.4, 0.2, 0.4)	0.0197	0.0867	9.4	0.450	no
(0.4, 0.4, 0.2)	0.0197	0.0867	9.4	0.450	no
(0.4, 0.6, 0.0)	0.0216	0.0928	9.4	0.450	no
(0.6, 0.0, 0.4)	0.0189	0.0865	9.4	0.450	no
(0.6, 0.2, 0.2)	0.0189	0.0865	9.4	0.450	no
(0.6, 0.4, 0.0)	0.0189	0.0865	9.4	0.450	no
(0.8, 0.0, 0.2)	0.0198	0.0867	9.4	0.450	no
(0.8, 0.2, 0.0)	0.0198	0.0867	9.4	0.450	no
(1.0, 0.0, 0.0)	**0.0189**	**0.0865**	**9.4**	**0.450**	yes

Over 21 points of the simplex, 2 are non-dominated and the front is convex (yes), so the weighted-sum scalarisation can in principle reach all of it — a caution worth verifying rather than assuming, since a non-convex front would leave part of itself unreachable by (28).

The relative excursions say which objectives actually respond: accuracy moves by 50.6% across the simplex, against 7.4% for effort and 4.3% for time. Only accuracy is genuinely under the control of the weights. Time and effort are nearly fixed, and Section 3.3.6 explains why: in path-parametrised mode the solver saturates $v_{x,\max}$ almost everywhere, so the duration of the mission is decided by the kinematics and the input bounds rather than by the tuning. The trade-off is real but mild, and the practical conclusion is that weight tuning is not where this system is limited — unlike the horizon, which Section 5.5 shows to be chosen badly.

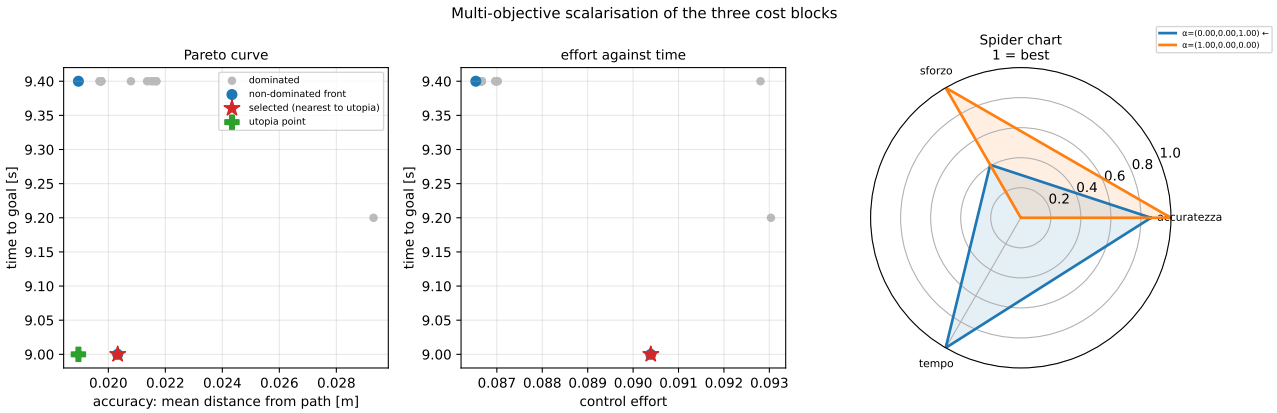

Figure 6: Sampling of the weight simplex: the achieved objectives for each scalarisation, the non-dominated set, and the utopia point. The metrics used to *evaluate* the solutions use fixed weights, not the sampled ones — otherwise each point of the simplex would be judged by a different yardstick.

5.8. The rolling-horizon local minimum, and its repair

The clearest illustration in this study of the difference between solving an optimization problem well and posing the right one does not involve the NLP at all. With a purely local grid and a memoryless target rule — the local goal taken as the intersection of the ray to the global goal with the grid boundary — the stack livelocks in a two-room world. The local target follows the ray into a partition wall; $A^\star$ correctly routes around the obstacle *inside the window*; the MPC tracks that path faithfully; and one grid width later the same rule sends the robot back, the memory of the wall having been erased with the grid. The result is a limit cycle: locally optimal at every cycle, globally useless. Every NLP in that run is solved to optimality.
Three structural facts conspire, and all three are properties of the *formulation*: the target rule is memoryless, the occupancy grid is volatile, and constraint (21c) forbids reversing, so the only escape from a concave pocket is a wide forward arc. Adding memory is therefore not a refinement but a requirement, and the repair must live in the target rule, which is where the failure lives.

Memory that survives noise. Every return is quantised to its cell in the *world* frame and stored with the time it was last seen and the number of *distinct scans* in which it has appeared; a cell is *confirmed* once that count reaches two. A wall is re-hit by every scan that ranges it, while a spurious return lands in its cell exactly once and never becomes an obstacle. Contradicting evidence is allowed to win as well: every beam passing *through* a remembered cell on its way to a farther return deletes it, stopping short of the endpoint so that a ray never carves the surface it terminates on. A real wall cannot be carved, because beams end on it; a phantom cell standing in an open passage is crossed, and erased, by every scan that ranges through it.

The re-posed target rule. With $\mathcal{G}_k$ a global grid built over the confirmed cells and π_k the $A^\star$ path from $\hat{x}$ to x_g on it, the local target becomes the point where the globally consistent route exits the local window,

$$x_{\text{local}} \;=\; \pi_k(s^\star), \qquad s^\star \;=\; \max\{\, s \;:\; \pi_k(s) \in \mathcal{W}(\hat{x}) \,\}. \tag{29}$$

The greedy rule is the empty-memory special case of (29): with no confirmed cells the global grid is free space, π_k is the straight segment to the goal, and its window exit *is* the ray–boundary intersection. The repair generalises the rule rather than replacing it.

Three hardening rules, each found by watching its absence fail. Confirmed cells are *removed from the search graph*, not merely inflated: under the soft cost (13) a remembered wall is only a finite obstacle, and an early version dutifully tunnelled through the partition the moment the detour became more expensive than the wall. Inflation prices *proximity*, memory records *evidence*, and only the latter may veto an edge. One-cell pinholes among confirmed cells are sealed in the routing graph, because at sensor range the beam spacing exceeds the cell size and a remembered wall carries single-cell gaps that open and close as scans accumulate — each a phantom shortcut that flips the global route. And the local execution graph dilates confirmed cells by the robot radius, so that no executed path passes within a body radius of known structure. The two graphs are deliberately not treated alike: dilating the routing graph makes the committed route invalidate itself as the observed wall grows under the robot's own approach, while sealing the execution graph after dilation counts

the body twice and closes every doorway the robot actually fits through. Stability lives in the routing graph, clearance in the execution graph.

5.9. Summary of design guidelines

Table 20 collects the actionable outcome of the campaigns above. Each row names the deployed choice, the choice the measurement supports, and where the evidence is.

Table 20: Design guidelines extracted from the numerical study. Bold in the second column marks a recommendation that differs from what is deployed.

Decision	Deployed / supported	Evidence
Integration scheme	mid-point / keep	Order 2.00 against 1.00, 200× more accurate for one addition (§3.2) — but see the next row before spending anything more here.
Prediction model	lag model / **revisit**	Divergence 0.305 m per horizon, 3505× the integrator error: the model, not the arithmetic, is the dominant term (§5.3).
Parametrisation	multiple shooting / keep	Hessian 4.45% dense against 100.00% condensed; claim the structure, not the speed (§3.3.1).
Obstacle relaxation	smooth penalty / keep, with ℓ^1 available	ℓ^1 slack exactly zero from $\rho_s = 1 \times 10^4 > \mu^\star$; ℓ^2 never vanishes (§3.3.3).
Obstacle at $k = 0$	excluded / keep	Including it cannot change the solution and makes the program infeasible exactly when it matters (§2.6, §3.3.3).
Terminal ingredient	none / **add**	Slack identically zero, cost 3.3% to 85.2%; but the result is flattered by the degenerate lag and must be re-measured under physics (§3.3.5).
Reference	time-parametrised / **path-parametrised**	+58% advance per horizon and one parameter eliminated, at +70% iterations (§3.3.6).
Derivatives	AD / keep	1.51 objective evaluations against 282, and more accurate; finite differences would cost 24% of the cycle budget (§4.2).
Hessian	exact / keep	20 against 36 iterations (−44%) at the same optimum (§4.2).
Solver class	interior point / keep	33.5× faster at 60 inequalities, 10.0× at 300 (§4.4).
Barrier weight W_{obs}	120 / keep	Below the bifurcation threshold $[200, 300]$: the control law is regular in x_0 (§5.4).
Horizon $N, \Delta t$	15, 0.20 s / **dominated**	$N = 5$, $\Delta t = 0.40$ s matches it at 23.6 ms against 60.3 ms; validate on real missions first (§5.5).
Control horizon N_c	$= N$ / **reduce**	Up to 2.1× cheaper at identical task performance (§5.6).
Move blocking	absent / keep absent	Measured reasons, not omission (§5.6).
Weight tuning	scalarisation / not critical	Only accuracy responds (50.6% against 4.3% on time): not where the system is limited (§5.7).
Input envelope	$v_{y,\max} = 0.02$ / **the real limit**	Critical cone contracts to 5; the tightened constraint and the ℓ^1 constraint both fail on $\mathcal{U}$, not on the data (§4.5, §3.3.3, §3.3.7).
Planner memory	on / required	Without it the rolling-horizon rule livelocks with every NLP solved correctly (§5.8).

6. Discussion and limitations

What the formulation guarantees, and what it does not. In its deployed configuration the NLP is always feasible, because the only constraints are the dynamics and the input box; for a real-time loop with no fallback trajectory this is a genuine robustness property, and it is why the penalty formulation was preferred to a constrained one. But there is no terminal set in that configuration and no Lyapunov argument, so nominal

asymptotic stability and recursive feasibility in the standard sense [6] are not established. Section 3.3.5 supplies the missing ingredient and prices it, and Section 3.3.3 supplies the exact relaxation that makes a constrained formulation survivable; assembling the two into a configuration that is simultaneously deployable and certified is the natural next step, and it is not free — Section 5.6 shows that the long prediction horizon a rigorous terminal ingredient would want costs closed-loop performance, not merely computation.

The weakest link is not the optimizer. Every campaign in Section 5 solved its NLP successfully, and Section 4.5 certifies those solutions as strict local minima with unique multipliers. The systematic failures of the system are elsewhere. One is a *modelling* failure: the prediction diverges from the plant by 0.305 m over a single horizon, and no amount of numerical care inside the program can repair a model that is a planar holonomic agent standing in for a walking humanoid. The other is a *formulation* failure: the greedy rolling-horizon target rule of Section 5.8 discards the topology of the free space, and the repair had to be posed in the target rule rather than in the NLP.

The input set is the binding constraint, and it is not in the cost. Three independent measurements converge on this. The critical cone contracts to 5 dimensions (§4.5); the ℓ^1-relaxed obstacle constraint remains infeasible at demanding clearances because no admissible input translates the body sideways (§3.3.3); and the constraint tightening is inert on the recorded run for the same reason (§3.3.7). A controller whose admissible set has been reduced to essentially one degree of freedom is not being limited by its objective, and effort spent on weights is effort spent on the wrong object.

Scope of the numbers, stated plainly. The plant used throughout is kinematic, and deliberately so: with the prediction model equal to the simulation model, the experiments measure the optimizer rather than the gait. The price is that every conclusion depending on the actuator pole is out of reach here — which is why the selection of the sampling time from the plant bandwidth, a standard piece of this analysis, is absent from this report and deferred until τ is identified under physics. Two further limits belong in the same paragraph. The closed-loop counterfactuals use synthetic worlds, one of which (the U-trap) grazes obstacles in every configuration and therefore cannot support any comparison of clearances. And the loop is closed through a look-ahead setpoint rather than by applying $u_0^\star$, which washes out fine differences between MPC solutions and is why the constraint tightening of Section 3.3.7 could be verified in the plan but not in the executed trajectory.

7. Conclusions

This report analysed a deployed map-free navigation stack as what it is numerically: an exact $A^\star$ search on a Gaussian occupancy grid, a 141-variable nonlinear program with a 1.73% dense constraint Jacobian, and a set of design decisions each of which has a defensible alternative in the theory.

The decisions that survived measurement are worth stating as such. Algorithmic differentiation costs 1.51 objective evaluations against 282 for central differences and is exact, which is what makes the exact Hessian affordable and buys 44% of the iterations. The interior-point method is faster than the active-set alternative in both regimes into which this problem can be put, and its margin widens with the number of inequalities exactly as expected. The deployed obstacle weight sits below the bifurcation threshold, so the control law is a regular function of the state. And the first- and second-order optimality conditions hold at every cycle examined, with unique multipliers.

Four results contradicted the design, and they are the ones that taught the most. The mid-point rule is 200× more accurate than the scheme it replaced and changes the closed loop by essentially nothing, because the prediction is limited by the model and not by its integration. The deployed horizon is dominated: a configuration a quarter of the size matches it on task performance at a third of the tail solve time. Making the path abscissa a decision variable buys 58% more progress per horizon and removes a hand-set parameter rather than replacing it with another. And the constraint that decides what the controller can do is not in the cost at all: with the lateral bound at 0.02 m/s and reversing forbidden, the critical cone contracts to 5 dimensions and the controller ends the mission executing the only motion still admissible.

The natural continuations follow directly. Identify the actuator time constant under physics, which unblocks both the sampling-time analysis and an honest re-measurement of the terminal constraint. Apply $u_0^\star$ directly instead of through a look-ahead setpoint, so that improvements provable in the plan become observable in the trajectory. Adopt the path-parametrised reference, whose cost is known and whose benefit is measured. And revisit the input envelope before the cost weights, because that is where the evidence points.

Reproducibility

Every number, table and figure in this report is generated from the deployed code by a single command and enters the document through macros and included files, not by transcription. The measurements were taken on commit `a54dfe30ec` of branch `G1_optimal_trajectory` (2026-08-24), with the profile `planner_params_g1.yaml`, the recorded run `industrial_plant_v3`, CasADi 3.7.2 and NumPy 1.26.4. The generator verifies the LaTeX it produces before writing it — balanced environments, table rows matching their column specification, mathematical tokens inside math mode, macros used in the text but never defined, and cross-references without a target — and it refuses to emit a document that fails any of those checks. Assertions in the text are conditioned on the data: where a fitted exponent does not reach its predicted value, where a Pareto front is not informative, or where a sweep contains no case that exhibits the effect under discussion, the generated text says so instead of reporting the expected conclusion. Re-running the campaign and recompiling updates the report; a parameter changed in the code and not in the text cannot go unnoticed, because there is no text to change.

References

[1] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[3] Juan Miguel Jimeno. CHAMP: Hierarchical controller for highly dynamic quadrupedal locomotion. `https://github.com/chvmp/champ`, 2020.

[4] Jongwoo Lee. Hierarchical controller for highly dynamic locomotion utilizing pattern modulation and impedance control: implementation on the MIT Cheetah robot. M.Sc. thesis, Massachusetts Institute of Technology, Department of Mechanical Engineering, Cambridge, MA, USA, 2013. `https://dspace.mit.edu/handle/1721.1/85490`.

[5] Jialong Li, Xuxin Cheng, Tianshu Huang, Shiqi Yang, Ri-Zhao Qiu, and Xiaolong Wang. AMO: Adaptive motion optimization for hyper-dexterous humanoid whole-body control. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025. `https://arxiv.org/abs/2505.03738`.

[6] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[7] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, USA, 1986.

[8] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017.

[9] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022. `https://proceedings.mlr.press/v164/rudin22a.html`.

[10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. `https://arxiv.org/abs/1707.06347`.

[11] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.